```
    .example.org = EXAMPLE.ORG
```

With the following lines being included in the `example.org` zone file:

```
_kerberos._udp      IN  SRV     01 00 88 kerberos.example.org.
_kerberos._tcp      IN  SRV     01 00 88 kerberos.example.org.
_kpasswd._udp       IN  SRV     01 00 464 kerberos.example.org.
_kerberos-adm._tcp  IN  SRV     01 00 749 kerberos.example.org.
_kerberos           IN  TXT     EXAMPLE.ORG
```



In order for clients to be able to find the Kerberos services, they *must* have either a fully configured /etc/krb5.conf or a minimally configured /etc/krb5.conf *and* a properly configured DNS server.

Next, create the Kerberos database which contains the keys of all principals (users and hosts) encrypted with a master password. It is not required to remember this password as it will be stored in /var/heimdal/m-key; it would be reasonable to use a 45-character random password for this purpose. To create the master key, run `kstash` and enter a password:

```
# kstash
```

The output should be similar to the following:

```
Master key: xxxxxxxxxxxxxxxxxxxxxxx
Verifying password - Master key: xxxxxxxxxxxxxxxxxxxxxxx
```

Once the master key has been created, the database should be initialized. The Kerberos administrative tool kadmin(8) can be used on the KDC in a mode that operates directly on the database, without using the kadmind(8) network service, as `kadmin -l`. This resolves the chicken-and-egg problem of trying to connect to the database before it is created. At the `kadmin` prompt, use `init` to create the realm's initial database:

```
# kadmin -l
kadmin> init EXAMPLE.ORG
Realm max ticket life [unlimited]:
```

Lastly, while still in `kadmin`, create the first principal using `add`. Stick to the default options for the principal for now, as these can be changed later with `modify`. Type `?` at the prompt to see the available options.

```
kadmin> add tillman
```

The output should be similar to the following:

```
Max ticket life [unlimited]:
Max renewable life [unlimited]:
Principal expiration time [never]:
Password expiration time [never]:
Attributes []:
Password: xxxxxxxx
Verifying password - Password: xxxxxxxx
```

Next, start the KDC services by running:

```
# service kdc start
# service kadmind start
```

While there will not be any kerberized daemons running at this point, it is possible to confirm that the KDC is functioning by obtaining a ticket for the principal that was just created:

```
% kinit tillman
```

The output should be similar to the following:

```
tillman@EXAMPLE.ORG's Password:
```

Confirm that a ticket was successfully obtained using `klist`:

```
% klist
```

The output should be similar to the following:

```
Credentials cache: FILE:/tmp/krb5cc_1001
    Principal: tillman@EXAMPLE.ORG

  Issued                Expires               Principal
Aug 27 15:37:58 2013  Aug 28 01:37:58 2013  krbtgt/EXAMPLE.ORG@EXAMPLE.ORG
```

The temporary ticket can be destroyed when the test is finished:

```
% kdestroy
```

### 16.9.2. Configuring a Server to Use Kerberos

The first step in configuring a server to use Kerberos authentication is to ensure that it has the correct configuration in /etc/krb5.conf. The version from the KDC can be used as-is, or it can be

regenerated on the new system.

Next, create /etc/krb5.keytab on the server. This is the main part of "Kerberizing" a service - it corresponds to generating a secret shared between the service and the KDC. The secret is a cryptographic key, stored in a "keytab". The keytab contains the server's host key, which allows it and the KDC to verify each others' identity. It must be transmitted to the server in a secure fashion, as the security of the server can be broken if the key is made public. Typically, the keytab is generated on an administrator's trusted machine using `kadmin`, then securely transferred to the server, e.g., with scp(1); it can also be created directly on the server if that is consistent with the desired security policy. It is very important that the keytab is transmitted to the server in a secure fashion: if the key is known by some other party, that party can impersonate any user to the server! Using `kadmin` on the server directly is convenient, because the entry for the host principal in the KDC database is also created using `kadmin`.

Of course, `kadmin` is a kerberized service; a Kerberos ticket is needed to authenticate to the network service, but to ensure that the user running `kadmin` is actually present (and their session has not been hijacked), `kadmin` will prompt for the password to get a fresh ticket. The principal authenticating to the kadmin service must be permitted to use the `kadmin` interface, as specified in /var/heimdal/kadmind.acl. See the section titled "Remote administration" in `info heimdal` for details on designing access control lists. Instead of enabling remote `kadmin` access, the administrator could securely connect to the KDC via the local console or ssh(1), and perform administration locally using `kadmin -l`.

After installing /etc/krb5.conf, use `add --random-key` in `kadmin`. This adds the server's host principal to the database, but does not extract a copy of the host principal key to a keytab. To generate the keytab, use `ext` to extract the server's host principal key to its own keytab:

```
# kadmin
```

The output should be similar to the following:

```
kadmin> add --random-key host/myserver.example.org
Max ticket life [unlimited]:
Max renewable life [unlimited]:
Principal expiration time [never]:
Password expiration time [never]:
Attributes []:
kadmin> ext_keytab host/myserver.example.org
kadmin> exit
```

Note that `ext_keytab` stores the extracted key in /etc/krb5.keytab by default. This is good when being run on the server being kerberized, but the `--keytab path/to/file` argument should be used when the keytab is being extracted elsewhere:

```
# kadmin
```

The output should be similar to the following:

```
kadmin> ext_keytab --keytab=/tmp/example.keytab host/myserver.example.org
kadmin> exit
```

The keytab can then be securely copied to the server using scp(1) or a removable media. Be sure to specify a non-default keytab name to avoid inserting unneeded keys into the system's keytab.

At this point, the server can read encrypted messages from the KDC using its shared key, stored in krb5.keytab. It is now ready for the Kerberos-using services to be enabled. One of the most common such services is sshd(8), which supports Kerberos via the GSS-API. In /etc/ssh/sshd_config, add the line:

```
GSSAPIAuthentication yes
```

After making this change, sshd(8) must be restarted for the new configuration to take effect: `service sshd restart`.

### 16.9.3. Configuring a Client to Use Kerberos

As it was for the server, the client requires configuration in /etc/krb5.conf. Copy the file in place (securely) or re-enter it as needed.

Test the client by using `kinit`, `klist`, and `kdestroy` from the client to obtain, show, and then delete a ticket for an existing principal. Kerberos applications should also be able to connect to Kerberos enabled servers. If that does not work but obtaining a ticket does, the problem is likely with the server and not with the client or the KDC. In the case of kerberized ssh(1), GSS-API is disabled by default, so test using `ssh -o GSSAPIAuthentication=yes hostname`.

When testing a Kerberized application, try using a packet sniffer such as `tcpdump` to confirm that no sensitive information is sent in the clear.

Various Kerberos client applications are available. With the advent of a bridge so that applications using SASL for authentication can use GSS-API mechanisms as well, large classes of client applications can use Kerberos for authentication, from Jabber clients to IMAP clients.

Users within a realm typically have their Kerberos principal mapped to a local user account. Occasionally, one needs to grant access to a local user account to someone who does not have a matching Kerberos principal. For example, `tillman@EXAMPLE.ORG` may need access to the local user account `webdevelopers`. Other principals may also need access to that local account.

The .k5login and .k5users files, placed in a user's home directory, can be used to solve this problem. For example, if the following .k5login is placed in the home directory of `webdevelopers`, both principals listed will have access to that account without requiring a shared password:

```
tillman@example.org
jdoe@example.org
```

Refer to ksu(1) for more information about .k5users.

### 16.9.4. MIT Differences

The major difference between the MIT and Heimdal implementations is that `kadmin` has a different, but equivalent, set of commands and uses a different protocol. If the KDC is MIT, the Heimdal version of `kadmin` cannot be used to administer the KDC remotely, and vice versa.

Client applications may also use slightly different command line options to accomplish the same tasks. Following the instructions at http://web.mit.edu/Kerberos/www/ is recommended. Be careful of path issues: the MIT port installs into /usr/local/ by default, and the FreeBSD system applications run instead of the MIT versions if `PATH` lists the system directories first.

When using MIT Kerberos as a KDC on FreeBSD, execute the following commands to add the required configurations to /etc/rc.conf:

```
# sysrc kdc_program="/usr/local/sbin/krb5kdc"
# sysrc kadmind_program="/usr/local/sbin/kadmind"
# sysrc kdc_flags=""
# sysrc kdc_enable="YES"
# sysrc kadmind_enable="YES"
```

### 16.9.5. Kerberos Tips, Tricks, and Troubleshooting

When configuring and troubleshooting Kerberos, keep the following points in mind:

- When using either Heimdal or MITKerberos from ports, ensure that the `PATH` lists the port's versions of the client applications before the system versions.
- If all the computers in the realm do not have synchronized time settings, authentication may fail. "Clock Synchronization with NTP" describes how to synchronize clocks using NTP.
- If the hostname is changed, the `host/` principal must be changed and the keytab updated. This also applies to special keytab entries like the `HTTP/` principal used for Apache's www/mod_auth_kerb.
- All hosts in the realm must be both forward and reverse resolvable in DNS or, at a minimum, exist in /etc/hosts. CNAMEs will work, but the A and PTR records must be correct and in place. The error message for unresolvable hosts is not intuitive: `Kerberos5 refuses authentication because Read req failed: Key table entry not found`.
- Some operating systems that act as clients to the KDC do not set the permissions for `ksu` to be setuid `root`. This means that `ksu` does not work. This is a permissions problem, not a KDC error.
- With MITKerberos, to allow a principal to have a ticket life longer than the default lifetime of ten hours, use `modify_principal` at the kadmin(8) prompt to change the `maxlife` of both the principal in question and the `krbtgt` principal. The principal can then use `kinit -l` to request a ticket with a longer lifetime.
- When running a packet sniffer on the KDC to aid in troubleshooting while running `kinit` from a workstation, the Ticket Granting Ticket (TGT) is sent immediately, even before the password is

typed. This is because the Kerberos server freely transmits a TGT to any unauthorized request. However, every TGT is encrypted in a key derived from the user's password. When a user types their password, it is not sent to the KDC, it is instead used to decrypt the TGT that `kinit` already obtained. If the decryption process results in a valid ticket with a valid time stamp, the user has valid Kerberos credentials. These credentials include a session key for establishing secure communications with the Kerberos server in the future, as well as the actual TGT, which is encrypted with the Kerberos server's own key. This second layer of encryption allows the Kerberos server to verify the authenticity of each TGT.

- Host principals can have a longer ticket lifetime. If the user principal has a lifetime of a week but the host being connected to has a lifetime of nine hours, the user cache will have an expired host principal and the ticket cache will not work as expected.
- When setting up krb5.dict to prevent specific bad passwords from being used as described in kadmind(8), remember that it only applies to principals that have a password policy assigned to them. The format used in krb5.dict is one string per line. Creating a symbolic link to /usr/share/dict/words might be useful.

### 16.9.6. Mitigating Kerberos Limitations

Since Kerberos is an all or nothing approach, every service enabled on the network must either be modified to work with Kerberos or be otherwise secured against network attacks. This is to prevent user credentials from being stolen and re-used. An example is when Kerberos is enabled on all remote shells but the non-Kerberized POP3 mail server sends passwords in plain text.

The KDC is a single point of failure. By design, the KDC must be as secure as its master password database. The KDC should have absolutely no other services running on it and should be physically secure. The danger is high because Kerberos stores all passwords encrypted with the same master key which is stored as a file on the KDC.

A compromised master key is not quite as bad as one might fear. The master key is only used to encrypt the Kerberos database and as a seed for the random number generator. As long as access to the KDC is secure, an attacker cannot do much with the master key.

If the KDC is unavailable, network services are unusable as authentication cannot be performed. This can be alleviated with a single master KDC and one or more slaves, and with careful implementation of secondary or fall-back authentication using PAM.

Kerberos allows users, hosts and services to authenticate between themselves. It does not have a mechanism to authenticate the KDC to the users, hosts, or services. This means that a trojaned `kinit` could record all user names and passwords. File system integrity checking tools like security/tripwire can alleviate this.

### 16.9.7. Resources and Further Information

- The Kerberos FAQ
- Designing an Authentication System: a Dialog in Four Scenes
- RFC 4120, The Kerberos Network Authentication Service (V5)
- MIT Kerberos home page

- Heimdal Kerberos project wiki page

# 16.10. TCP Wrappers

TCP Wrappers is a host-based network access control system. By intercepting incoming network requests before they reach the actual network service, TCP Wrappers assess whether the source IP address is permitted or denied access based on predefined rules in configuration files.

However, while TCP Wrappers provide basic access control, they should not be considered a substitute for more robust security measures. For comprehensive protection, it's recommended to use advanced technologies like firewalls, along with proper user authentication practices and intrusion detection systems.

## 16.10.1. Initial Configuration

TCP Wrappers are enabled by default in inetd(8). So the first step will be to enable inetd(8) executing the following commands:

```
# sysrc inetd_enable="YES"
# service inetd start
```

Then, properly configure /etc/hosts.allow.



Unlike other implementations of TCP Wrappers, the use of hosts.deny is deprecated in FreeBSD. All configuration options should be placed in /etc/hosts.allow.

In the simplest configuration, daemon connection policies are set to either permit or block, depending on the options in /etc/hosts.allow. The default configuration in FreeBSD is to allow all connections to the daemons started with inetd.

Basic configuration usually takes the form of `daemon : address : action`, where `daemon` is the daemon which inetd started, `address` is a valid hostname, IP address, or an IPv6 address enclosed in brackets ([ ]), and `action` is either `allow` or `deny`. TCP Wrappers uses a first rule match semantic, meaning that the configuration file is scanned from the beginning for a matching rule. When a match is found, the rule is applied and the search process stops.

For example, to allow POP3 connections via the mail/qpopper daemon, the following lines should be appended to /etc/hosts.allow:

```
# This line is required for POP3 connections:
qpopper : ALL : allow
```

Whenever this file is edited, restart inetd:

```
# service inetd restart
```

### 16.10.2. Advanced Configuration

TCP Wrappers provides advanced options to allow more control over the way connections are handled. In some cases, it may be appropriate to return a comment to certain hosts or daemon connections. In other cases, a log entry should be recorded or an email sent to the administrator. Other situations may require the use of a service for local connections only. This is all possible through the use of configuration options known as wildcards, expansion characters, and external command execution. To learn more about wildcards and their associated functionality, refer to hosts_access(5).

## 16.11. Access Control Lists

Access Control Lists (ACLs) extend traditional UNIX® file permissions by allowing fine-grained access control for users and groups on a per-file or per-directory basis. Each ACL entry defines a user or group and the associated permissions, such as read, write, and execute. FreeBSD provides commands like getfacl(1) and setfacl(1) to manage ACLs.

ACLs are useful in scenarios requiring more specific access control than standard permissions, commonly used in multi-user environments or shared hosting. However, complexity may be unavoidable, but careful planning is required to ensure that the desired security properties are being provided



FreeBSD supports the implementation of NFSv4 ACLs in both UFS and OpenZFS. Please note that some arguments to the setfacl(1) command only work with POSIX ACLs and others in NFSv4 ACLs.

### 16.11.1. Enabling ACL Support in UFS

ACLs are enabled by the mount-time administrative flag, `acls`, which may be added to /etc/fstab.

Therefore it will be necessary to access /etc/fstab and in the options section add the `acls` flag as follows:

```
# Device        Mountpoint      FStype    Options     Dump      Pass#
/dev/ada0s1a    /               ufs       rw,acls     1         1
```

### 16.11.2. Get ACLs information

It is possible to check the ACLs of a file or a directory using getfacl(1).

For example, to view the ACL settings on ~/test file execute the following command:

```
% getfacl test
```

The output should be similar to the following in case of using NFSv4 ACLs:

```
# file: test
# owner: freebsduser
# group: freebsduser
           owner@:rw-p--aARWcCos:-------:allow
           group@:r-----a-R-c--s:-------:allow
        everyone@:r-----a-R-c--s:-------:allow
```

And the output should be similar to the following in case of using POSIX.1e ACLs:

```
# file: test
# owner: freebsduser
# group: freebsduser
user::rw-
group::r--
other::r--
```

### 16.11.3. Working with ACLs

setfacl(1) can be used to add, modify or remove ACLs from a file or directory.

As noted above, some arguments to setfacl(1) do not work with NFSv4 ACLs, and vice versa. This section covers how to execute the commands for POSIX ACLs and for NFSv4 ACLs and shows examples of both.

For example, to set the mandatory elements of the POSIX.1e default ACL:

```
% setfacl -d -m u::rwx,g::rx,o::rx,mask::rwx directory
```

This other example sets read, write, and execute permissions for the file owner's POSIX.1e ACL entry and read and write permissions for group mail on file:

```
% setfacl -m u::rwx,g:mail:rw file
```

To do the same as in the previous example but in NFSv4 ACL:

```
% setfacl -m owner@:rwxp::allow,g:mail:rwp::allow file
```

To remove all ACL entries except for the three required from file in POSIX.1e ACL:

```
% setfacl -bn file
```

To remove all ACL entries in NFSv4 ACL:

```
% setfacl -b file
```

Refer to getfacl(1) and setfacl(1) for more information about the options available for these commands.

# 16.12. Capsicum

Capsicum is a lightweight OS capability and sandbox framework implementing a hybrid capability system model. Capabilities are unforgeable tokens of authority that can be delegated and must be presented to perform an action. Capsicum makes file descriptors into capabilities.

Capsicum can be used for application and library compartmentalisation, the decomposition of larger bodies of software into isolated (sandboxed) components in order to implement security policies and limit the impact of software vulnerabilities.

# 16.13. Process Accounting

Process accounting is a security method in which an administrator may keep track of system resources used and their allocation among users, provide for system monitoring, and minimally track a user's commands.

Process accounting has both positive and negative points. One of the positives is that an intrusion may be narrowed down to the point of entry. A negative is the amount of logs generated by process accounting, and the disk space they may require. This section walks an administrator through the basics of process accounting.



If more fine-grained accounting is needed, refer to Security Event Auditing.

## 16.13.1. Enabling and Utilizing Process Accounting

Before using process accounting, it must be enabled using the following commands:

```
# sysrc accounting_enable=yes
# service accounting start
```

The accounting information is stored in files located in /var/account, which is automatically created, if necessary, the first time the accounting service starts. These files contain sensitive information, including all the commands issued by all users. Write access to the files is limited to `root`, and read access is limited to `root` and members of the `wheel` group. To also prevent members of `wheel` from reading the files, change the mode of the /var/account directory to allow access only by `root`.

Once enabled, accounting will begin to track information such as CPU statistics and executed commands. All accounting logs are in a non-human readable format which can be viewed using

sa(8). If issued without any options, sa(8) prints information relating to the number of per-user calls, the total elapsed time in minutes, total CPU and user time in minutes, and the average number of I/O operations. Refer to sa(8) for the list of available options which control the output.

To display the commands issued by users, use `lastcomm`.

For example, this command prints out all usage of `ls` by `trhodes` on the `ttyp1` terminal:

```
# lastcomm ls trhodes ttyp1
```

Many other useful options exist and are explained in lastcomm(1), acct(5), and sa(8).

# 16.14. Resource Limits

In FreeBSD, resource limits refer to the mechanisms that control and manage the allocation of various system resources to processes and users. These limits are designed to prevent a single process or user from consuming an excessive amount of resources, which could lead to performance degradation or system instability. Resource limits help ensure fair resource distribution among all active processes and users on the system.

FreeBSD provides several methods for an administrator to limit the amount of system resources an individual may use.

The traditional method defines login classes by editing /etc/login.conf. While this method is still supported, any changes require a multi-step process of editing this file, rebuilding the resource database, making necessary changes to /etc/master.passwd, and rebuilding the password database. This can become time consuming, depending upon the number of users to configure.

rctl(8) can be used to provide a more fine-grained method for controlling resource limits. This command supports more than user limits as it can also be used to set resource constraints on processes and jails.

This section demonstrates both methods for controlling resources, beginning with the traditional method.

## 16.14.1. Types of Resources

FreeBSD provides limits for various types of resources, including:

*Table 30. Resource types*

| Type | Description |
|---|---|
| CPU Time | Limits the amount of CPU time a process can consume |
| Memory | Controls the amount of physical memory a process can use |

| Type | Description |
| --- | --- |
| Open Files | Limits the number of files a process can have open simultaneously |
| Processes | Controls the number of processes a user or a process can create |
| File Size | Limits the maximum size of files that a process can create |
| Core Dumps | Controls whether processes are allowed to generate core dump files |
| Network Resources | Limits the amount of network resources (e.g., sockets) a process can use |

For a full listing of types see login.conf(5) and rctl(8).

### 16.14.2. Configuring Login Classes

In the traditional method, login classes and the resource limits to apply to a login class are defined in /etc/login.conf. Each user account can be assigned to a login class, where `default` is the default login class. Each login class has a set of login capabilities associated with it. A login capability is a `name=value` pair, where *name* is a well-known identifier and *value* is an arbitrary string which is processed accordingly depending on the *name*.

The first step to configure a resource limit will be to open /etc/login.conf by executing the following command:

```
# ee /etc/login.conf
```

Then locate the section for the user class to be modified. In this example, let's assume the user class is named `limited`, create it in case it does not exist.

```
limited:\ ①
        :maxproc=50:\ ②
        :tc=default: ③
```

① Name of the user class.

② Sets the maximum number of processes (maxproc) to 50 for users in the `limited` class.

③ Indicates that this user class inherits the default settings from the "default" class.

After modifying the /etc/login.conf file, run cap_mkdb(1) to generate the database that FreeBSD uses to apply these settings:

```
# cap_mkdb /etc/login.conf
```

chpass(1) can be used to change the class to the desired user by executing the following command:

```
# chpass username
```

This will open a text editor, add the new `limited` class there as follows:

```
#Changing user information for username.
Login: username
Password: $6$2H.419USdGaiJeqK$6kgcTnDadasdasd3YnlNZsOni5AMymibkAfRCPirc7ZFjjv
DVsKyXx26daabdfqSdasdsmL/ZMUpdHiO0
Uid [#]: 1001
Gid [# or name]: 1001
Change [month day year]:
Expire [month day year]:
Class: limited
Home directory: /home/username
Shell: /bin/sh
Full Name: User &
Office Location:
Office Phone:
Home Phone:
Other information:
```

Now, the user assigned to the `limited` class will have a maximum process limit of 50. Remember that this is just one example of setting a resource limit using the /etc/login.conf file.

Keep in mind that after making changes to the /etc/login.conf file, the user needs to log out and log back in for the changes to take effect. Additionally, always exercise caution when editing system configuration files, especially when using privileged access.

### 16.14.3. Enabling and Configuring Resource Limits

The rctl(8) system provides a more fine-grained way to set and manage resource limits for individual processes and users. It allows dynamically assigning resource limits to specific processes or users, regardless of their user class.

The first step to use rctl(8) will be to enable it adding the following line to /boot/loader.conf and reboot the system:

```
kern.racct.enable=1
```

Then enable and start the rctl(8) service by executing the following commands:

```
# sysrc rctl_enable="YES"
# service rctl start
```

Then rctl(8) may be used to set rules for the system.

Rule syntax (rctl.conf(5)) is controlled through the use of a subject, subject-id, resource, and action, as seen in this example rule:

```
subject:subject-id:resource:action=amount/per
```

For example to constrained the user to add no more than 10 processes execute the following command:

```
# rctl -a user:username:maxproc:deny=10/user
```

To check the applied resource limits the rctl(8) command can be executed:

```
# rctl
```

The output should be similar to the following:

```
user:username:maxproc:deny=10
```

Rules will persist across reboots if they have been added to /etc/rctl.conf. The format is a rule, without the preceding command. For example, the previous rule could be added as:

```
user:username:maxproc:deny=10
```

## 16.15. Monitoring Third Party Security Issues

In recent years, the security world has made many improvements to how vulnerability assessment is handled. The threat of system intrusion increases as third party utilities are installed and configured for virtually any operating system available today.

Vulnerability assessment is a key factor in security. While FreeBSD releases advisories for the base system, doing so for every third party utility is beyond the FreeBSD Project's capability. There is a way to mitigate third party vulnerabilities and warn administrators of known security issues. A FreeBSD add on utility known as pkg includes options explicitly for this purpose.

pkg polls a database for security issues. The database is updated and maintained by the FreeBSD Security Team and ports developers.

Installation provides periodic(8) configuration files for maintaining the pkg audit database, and provides a programmatic method of keeping it updated.

After installation, and to audit third party utilities as part of the Ports Collection at any time, an administrator may choose to update the database and view known vulnerabilities of installed

packages by invoking:

```
% pkg audit -F
```

The output should be similar to the following:

```
vulnxml file up-to-date
chromium-116.0.5845.96_1 is vulnerable:
  chromium -- multiple vulnerabilities
  CVE: CVE-2023-4431
  CVE: CVE-2023-4427
  CVE: CVE-2023-4428
  CVE: CVE-2023-4429
  CVE: CVE-2023-4430
  WWW: https://vuxml.FreeBSD.org/freebsd/5fa332b9-4269-11ee-8290-a8a1599412c6.html

samba413-4.13.17_5 is vulnerable:
  samba -- multiple vulnerabilities
  CVE: CVE-2023-3347
  CVE: CVE-2023-34966
  CVE: CVE-2023-34968
  CVE: CVE-2022-2127
  CVE: CVE-2023-34967
  WWW: https://vuxml.FreeBSD.org/freebsd/441e1e1a-27a5-11ee-a156-080027f5fec9.html

2 problem(s) in 2 installed package(s) found.
```

By pointing a web browser to the displayed URL, an administrator may obtain more information about the vulnerability.

This will include the versions affected, by FreeBSD port version, along with other web sites which may contain security advisories.

## 16.16. FreeBSD Security Advisories

Like many producers of quality operating systems, the FreeBSD Project has a security team which is responsible for determining the End-of-Life (EoL) date for each FreeBSD release and to provide security updates for supported releases which have not yet reached their EoL. More information about the FreeBSD security team and the supported releases is available on the FreeBSD security page.

One task of the security team is to respond to reported security vulnerabilities in the FreeBSD operating system. Once a vulnerability is confirmed, the security team verifies the steps necessary to fix the vulnerability and updates the source code with the fix. It then publishes the details as a "Security Advisory". Security advisories are published on the FreeBSD website and mailed to the FreeBSD security notifications mailing list, FreeBSD security mailing list, and FreeBSD announcements mailing list.

### 16.16.1. Format of a Security Advisory

Here is an example of a FreeBSD security advisory:

```
-----BEGIN PGP SIGNED MESSAGE-----
Hash: SHA512

=============================================================================
FreeBSD-SA-23:07.bhyve                                      Security Advisory
                                                          The FreeBSD Project

Topic:          bhyve privileged guest escape via fwctl

Category:       core
Module:         bhyve
Announced:      2023-08-01
Credits:        Omri Ben Bassat and Vladimir Eli Tokarev from Microsoft
Affects:        FreeBSD 13.1 and 13.2
Corrected:      2023-08-01 19:48:53 UTC (stable/13, 13.2-STABLE)
                2023-08-01 19:50:47 UTC (releng/13.2, 13.2-RELEASE-p2)
                2023-08-01 19:48:26 UTC (releng/13.1, 13.1-RELEASE-p9)
CVE Name:       CVE-2023-3494

For general information regarding FreeBSD Security Advisories,
including descriptions of the fields above, security branches, and the
following sections, please visit <URL:https://security.FreeBSD.org/>.

I.   Background

bhyve(8)'s fwctl interface provides a mechanism through which guest
firmware can query the hypervisor for information about the virtual
machine.  The fwctl interface is available to guests when bhyve is run
with the "-l bootrom" option, used for example when booting guests in
UEFI mode.

bhyve is currently only supported on the amd64 platform.

II.  Problem Description

The fwctl driver implements a state machine which is executed when the
guest accesses certain x86 I/O ports.  The interface lets the guest copy
a string into a buffer resident in the bhyve process' memory.  A bug in
the state machine implementation can result in a buffer overflowing when
copying this string.

III. Impact

A malicious, privileged software running in a guest VM can exploit the
buffer overflow to achieve code execution on the host in the bhyve
userspace process, which typically runs as root.  Note that bhyve runs
```

```
in a Capsicum sandbox, so malicious code is constrained by the
capabilities available to the bhyve process.

IV.  Workaround

No workaround is available.  bhyve guests that are executed without the
"-l bootrom" option are unaffected.

V.   Solution

Upgrade your vulnerable system to a supported FreeBSD stable or
release / security branch (releng) dated after the correction date.

Perform one of the following:

1) To update your vulnerable system via a binary patch:

Systems running a RELEASE version of FreeBSD on the amd64, i386, or
(on FreeBSD 13 and later) arm64 platforms can be updated via the
freebsd-update(8) utility:

# freebsd-update fetch
# freebsd-update install

Restart all affected virtual machines.

2) To update your vulnerable system via a source code patch:

The following patches have been verified to apply to the applicable
FreeBSD release branches.

a) Download the relevant patch from the location below, and verify the
detached PGP signature using your PGP utility.

[FreeBSD 13.2]
# fetch https://security.FreeBSD.org/patches/SA-23:07/bhyve.13.2.patch
# fetch https://security.FreeBSD.org/patches/SA-23:07/bhyve.13.2.patch.asc
# gpg --verify bhyve.13.2.patch.asc

[FreeBSD 13.1]
# fetch https://security.FreeBSD.org/patches/SA-23:07/bhyve.13.1.patch
# fetch https://security.FreeBSD.org/patches/SA-23:07/bhyve.13.1.patch.asc
# gpg --verify bhyve.13.1.patch.asc

b) Apply the patch.  Execute the following commands as root:

# cd /usr/src
# patch < /path/to/patch

c) Recompile the operating system using buildworld and installworld as
described in <URL:https://www.FreeBSD.org/handbook/makeworld.html>.
```

```
Restart all affected virtual machines.

VI.  Correction details

This issue is corrected by the corresponding Git commit hash or Subversion
revision number in the following stable and release branches:

Branch/path                             Hash                     Revision
- -------------------------------------------------------------------------
stable/13/                              9fe302d78109    stable/13-n255918
releng/13.2/                            2bae613e0da3  releng/13.2-n254625
releng/13.1/                            87702e38a4b4  releng/13.1-n250190
- -------------------------------------------------------------------------

Run the following command to see which files were modified by a
particular commit:

# git show --stat <commit hash>

Or visit the following URL, replacing NNNNNN with the hash:

<URL:https://cgit.freebsd.org/src/commit/?id=NNNNNN>

To determine the commit count in a working tree (for comparison against
nNNNNNN in the table above), run:

# git rev-list --count --first-parent HEAD

VII. References

<URL:https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2023-3494>

The latest revision of this advisory is available at
<URL:https://security.FreeBSD.org/advisories/FreeBSD-SA-23:07.bhyve.asc>
-----BEGIN PGP SIGNATURE-----

iQIzBAEBCgAdFiEEthUnfoEIffdcgYM7bljekB8AGu8FAmTJdsIACgkQbljekB8A
Gu8Q1Q/7BFw5Aa0cFxBzbdz+O5NAImj58MvKS6xw61bXcYr12jchyT6ENC7yiR+K
qCqbe5TssRbtZ1gg/94gSGEXccz5OcJGxW+qozhcdPUh2L2nzBPkMCrclrYJfTtM
cnmQKjg/wFZLUVr71GEM95ZFaktlZdXyXx9Z8eBzow5rXexpl1TTHQQ2kZZ41K4K
KFhup91dzGCIj02cqbl+1h5BrXJe3s/oNJt5JKIh/GBh5THQu9n6AywQYl18HtjV
fMb1qRTAS9WbiEP5QV2eEuOG86ucuhytqnEN5MnXJ2rLSjfb9izs9HzLo3ggy7yb
hN3tlbfIPjMEwYexieuoyP3rzKkLeYfLXqJU4zKCRnIbBIkMRy4mcFkfcYmI+MhF
NPh2R9kccemppKXeDhKJurH0vsetr8ti+AwOZ3pgO21+9w+mjE+EfaedIi+JWhip
hwqeFv03bAQHJdacNYGV47NsJ91CY4ZgWC3ZOzBZ2Y5SDtKFjyc0bf83WTfU9A/0
drC0z3xaJribah9e6k5d7lmZ7L6aHCbQ70+aayuAEZQLr/N1doB0smNi0IHdrtY0
JdIqmVX+d1ihVhJ05prC460AS/KolqiaysunligxR+ZnctE9Xdo1BlLEbYu2KjT4
LpWvSuhRMSQaYkJU72SodQc0FM5mqqNN42Vx+X4EutOfvQuRGlI=
=MlAY
```

```
-----END PGP SIGNATURE-----
```

Every security advisory uses the following format:

- Each security advisory is signed by the PGP key of the Security Officer. The public key for the Security Officer can be verified at OpenPGP Keys.
- The name of the security advisory always begins with `FreeBSD-SA-` (for FreeBSD Security Advisory), followed by the year in two digit format (`23:`), followed by the advisory number for that year (`07.`), followed by the name of the affected application or subsystem (`bhyve`).
- The `Topic` field summarizes the vulnerability.
- The `Category` refers to the affected part of the system which may be one of `core`, `contrib`, or `ports`. The `core` category means that the vulnerability affects a core component of the FreeBSD operating system. The `contrib` category means that the vulnerability affects software included with FreeBSD, such as BIND. The `ports` category indicates that the vulnerability affects software available through the Ports Collection.
- The `Module` field refers to the component location. In this example, the `bhyve` module is affected; therefore, this vulnerability affects an application installed with the operating system.
- The `Announced` field reflects the date the security advisory was published. This means that the security team has verified that the problem exists and that a patch has been committed to the FreeBSD source code repository.
- The `Credits` field gives credit to the individual or organization who noticed the vulnerability and reported it.
- The `Affects` field explains which releases of FreeBSD are affected by this vulnerability.
- The `Corrected` field indicates the date, time, time offset, and releases that were corrected. The section in parentheses shows each branch for which the fix has been merged, and the version number of the corresponding release from that branch. The release identifier itself includes the version number and, if appropriate, the patch level. The patch level is the letter `p` followed by a number, indicating the sequence number of the patch, allowing users to track which patches have already been applied to the system.
- The `CVE Name` field lists the advisory number, if one exists, in the public cve.mitre.org security vulnerabilities database.
- The `Background` field provides a description of the affected module.
- The `Problem Description` field explains the vulnerability. This can include information about the flawed code and how the utility could be maliciously used.
- The `Impact` field describes what type of impact the problem could have on a system.
- The `Workaround` field indicates if a workaround is available to system administrators who cannot immediately patch the system.
- The `Solution` field provides the instructions for patching the affected system. This is a step by step tested and verified method for getting a system patched and working securely.
- The `Correction Details` field displays each affected Subversion or Git branch with the revision number that contains the corrected code.

- The `References` field offers sources of additional information regarding the vulnerability.

# Chapter 17. Jails and Containers

## 17.1. Synopsis

Since system administration is a difficult task, many tools have been developed to make life easier for the administrator. These tools often enhance the way systems are installed, configured, and maintained. One of the tools which can be used to enhance the security of a FreeBSD system is *jails*. Jails have been available since FreeBSD 4.X and continue to be enhanced in their usefulness, performance, reliability, and security.

Jails build upon the chroot(2) concept, which is used to change the root directory of a set of processes. This creates a safe environment, separate from the rest of the system. Processes created in the chrooted environment can not access files or resources outside of it. For that reason, compromising a service running in a chrooted environment should not allow the attacker to compromise the entire system.

However, a chroot has several limitations. It is suited to easy tasks which do not require much flexibility or complex, advanced features. Over time, many ways have been found to escape from a chrooted environment, making it a less than ideal solution for securing services.

Jails improve on the concept of the traditional chroot environment in several ways.

In a traditional chroot environment, processes are only limited in the part of the file system they can access. The rest of the system resources, system users, running processes, and the networking subsystem are shared by the chrooted processes and the processes of the host system. Jails expand this model by virtualizing access to the file system, the set of users, and the networking subsystem. More fine-grained controls are available for tuning the access of a jailed environment. Jails can be considered as a type of operating system-level virtualization.

This chapter covers:

- What a jail is and what purpose it may serve in FreeBSD installations.
- The different types of jail.
- The different ways to configure the network for a jail.
- The jail configuration file.
- How to create the different types of jail.
- How to start, stop, and restart a jail.
- The basics of jail administration, both from inside and outside the jail.
- How to upgrade the different types of jail.
- An incomplete list of the different FreeBSD jail managers.

## 17.2. Jail Types

Some administrators divide jails into different types, although the underlying technology is the same. Each administrator will have to assess what type of jail to create in each case depending on

the problem they have to solve.

Below can be found a list of the different types, their characteristics, and considerations for use.

### 17.2.1. Thick Jails

A thick jail is a traditional form of FreeBSD Jail. In a thick jail, a complete copy of the base system is replicated within the jail's environment. This means that the jail has its own separate instance of the FreeBSD base system, including libraries, executables, and configuration files. The jail can be thought of as an almost complete standalone FreeBSD installation, but running within the confines of the host system. This isolation ensures that the processes within the jail are kept separate from those on the host and other jails.

Advantages of Thick Jails:

- High degree of isolation: Processes within the jail are isolated from the host system and other jails.
- Independence: Thick jails can have different versions of libraries, configurations, and software than the host system or other jails.
- Security: Since the jail contains its own base system, vulnerabilities or issues affecting the jail environment will not directly impact the host or other jails.

Disadvantages of Thick Jails:

- Resource overhead: Because each jail maintains its own separate base system, thick jails consume more resources compared to thin jails.
- Maintenance: Each jail requires its own maintenance and updates for its base system components.

### 17.2.2. Thin Jails

A thin jail shares the base system using OpenZFS snapshots or NullFS mounts from a template. Only a minimal subset of base system is duplicated for each thin jail, resulting in less resource consumption compared to a thick jail. However, this also means that thin jails have less isolation and independence compared to thick jails. Changes in shared components could potentially affect multiple thin jails simultaneously.

In summary, a FreeBSD Thin Jail is a type of FreeBSD Jail that replicates a substantial portion, but not all, of the base system within the isolated environment.

Advantages of Thin Jails:

- Resource Efficiency: Thin jails are more resource-efficient compared to thick jails. Since they share most of the base system, they consume less disk space and memory. This makes it possible to run more jails on the same hardware without consuming excessive resources.
- Faster Deployment: Creating and launching thin jails is generally faster compared to thick jails. This can be particularly advantageous when rapidly deploying multiple instances.
- Unified Maintenance: Since thin jails share the majority of their base system with the host

system, updates and maintenance of common base system components (such as libraries and binaries) only need to be done once on the host. This simplifies the maintenance process compared to maintaining an individual base system for each thick jail.

- Shared Resources: Thin jails can more easily share common resources such as libraries and binaries with the host system. This can potentially lead to more efficient disk caching and improved performance for applications within the jail.

Disadvantages of Thin Jails:

- Reduced Isolation: The primary disadvantage of thin jails is that they offer less isolation compared to thick jails. Since they share a significant portion of the template's base system, vulnerabilities or issues affecting shared components could potentially impact multiple jails simultaneously.
- Security Concerns: The reduced isolation in thin jails could pose security risks, as a compromise in one jail might have a greater potential to affect other jails or the host system.
- Dependency Conflicts: If multiple thin jails require different versions of the same libraries or software, managing dependencies can become complex. In some cases, this might require additional effort to ensure compatibility.
- Compatibility Challenges: Applications within a thin jail might encounter compatibility issues if they assume a certain base system environment that differs from the shared components provided by the template.

### 17.2.3. Service Jails

A service jail shares the complete filesystem tree directly with the host (the jail root path is /) and as such can access and modify any file on the host, and shares the same user accounts with the host. By default it has no access to the network or other resources which are restricted in jails, but they can be configured to re-use the network of the host and to remove some of the jail-restrictions. The use case for service jails is automatic confinement of services/daemons inside a jail with minimal configuration, and without any knowledge of the files needed by such service/daemon. Service jails exist since FreeBSD 15.

Advantages of Service Jails:

- Zero Administration: A service jail ready service needs only one config line in /etc/rc.conf, a service which is not service jails ready needs two config lines.
- Resource Efficiency: Service jails are more resource efficient than thin jails, as they do not need any additional disk space or network resource.
- Faster Deployment: Creating and launching service jails is generally faster compared to thin jails if only distinct services/daemons shall be jailed and no parallel instances of the same service/daemon is needed.
- Shared Resources: Service jails share all resources such as libraries and binaries with the host system. This can potentially lead to more efficient disk caching and improved performance for applications within the jail.
- Process Isolation: Service jails isolate a particular service, it can not see processes which are not a child of the service jail, even if they run within the same user account.

Disadvantages of Service Jails:

- Reduced Isolation: The primary disadvantage of service jails is that they offer no filesystem isolation compared to thick or thin jails.
- Security Concerns: The reduced isolation in service jails could pose security risks, as a compromise in one jail might have a greater potential to affect everything on the host system.

Most of the configuration of jails which is discussed below is not needed for service jails. To understand how jails work, it is recommended to understand those configuration possibilities. The details about what is needed to configure a service jail is in Configuring service jails.

### 17.2.4. VNET Jails

A FreeBSD VNET jail is a virtualized environment that allows for the isolation and control of network resources for processes running within it. It provides a high level of network segmentation and security by creating a separate network stack for processes within the jail, ensuring that network traffic within the jail is isolated from the host system and other jails.

In essence, FreeBSD VNET jails add a network configuration mechanism. This means a VNET jail can be created as a Thick or Thin Jail.

### 17.2.5. Linux Jails

A FreeBSD Linux Jail is a feature in the FreeBSD operating system that enables the use of Linux binaries and applications within a FreeBSD jail. This functionality is achieved by incorporating a compatibility layer that allows certain Linux system calls and libraries to be translated and executed on the FreeBSD kernel. The purpose of a Linux Jail is to facilitate the execution of Linux software on a FreeBSD system without needing a separate Linux virtual machine or environment.

## 17.3. Host Configuration

Before creating any jail on the host system it is necessary to perform certain configuration and obtain some information from the host system.

It will be necessary to configure the jail(8) utility, create the necessary directories to configure and install jails, obtain information from the host's network, and check whether the host uses OpenZFS or UFS as its file system.



The FreeBSD version running in the jail can not be newer than the version running in the host.

### 17.3.1. Jail Utility

The jail(8) utility manages jails.

To start jails when the system boots, run the following commands:

```
# sysrc jail_enable="YES"
```

```
# sysrc jail_parallel_start="YES"
```



With `jail_parallel_start`, all configured jails will be started in the background.

### 17.3.2. Networking

Networking for FreeBSD jails can be configured several different ways:

**Host Networking Mode (IP Sharing)**

In host networking mode, a jail shares the same networking stack as the host system. When a jail is created in host networking mode it uses the same network interface and IP address. This means that the jail does not have a separate IP address, and its network traffic is associated with the host's IP.

**Virtual Networks (VNET)**

Virtual Networks are a feature of FreeBSD jails that offer more advanced and flexible networking solutions than a basic networking mode like host networking. VNET allows the creation of isolated network stacks for each jail, providing them with their own separate IP addresses, routing tables, and network interfaces. This offers a higher level of network isolation and allows jails to function as if they are running on separate virtual machines.

**The netgraph system**

netgraph(4) is a versatile kernel framework for creating custom network configurations. It can be used to define how network traffic flows between jails and the host system and between different jails.

### 17.3.3. Setting Up the Jail Directory Tree

There is no specific place to put the files for the jails.

Some administrators use /jail, others /usr/jail, and still others /usr/local/jails. In this chapter /usr/local/jails will be used.

Apart from /usr/local/jails other directories will be created:

- media will contain the compressed files of the downloaded userlands.
- templates will contain the templates when using Thin Jails.
- containers will contain the jails.

When using OpenZFS, execute the following commands to create datasets for these directories:

```
# zfs create -o mountpoint=/usr/local/jails zroot/jails
# zfs create zroot/jails/media
# zfs create zroot/jails/templates
# zfs create zroot/jails/containers
```



In this case, `zroot` was used for the parent dataset, but other datasets could have been used.

When using UFS, execute the following commands to create the directories:

```
# mkdir /usr/local/jails/
# mkdir /usr/local/jails/media
# mkdir /usr/local/jails/templates
# mkdir /usr/local/jails/containers
```

### 17.3.4. Jail Configuration Files

There are two ways to configure jails.

The first one is to add an entry for each jail to the file /etc/jail.conf. The other option is to create a file for each jail in the directory /etc/jail.conf.d/.

In case a host system has few jails, an entry for each jail can be added in the file /etc/jail.conf. If the host system has many jails, it is a good idea to have one configuration file for each jail in the /etc/jail.conf.d/ directory.

The files in /etc/jail.conf.d/ must have `.conf` as their extension and have to be included in /etc/jail.conf:

```
.include "/etc/jail.conf.d/*.conf";
```

A typical jail entry would look like this:

```
jailname { ①
  # STARTUP/LOGGING
  exec.start = "/bin/sh /etc/rc"; ②
  exec.stop = "/bin/sh /etc/rc.shutdown"; ③
  exec.consolelog = "/var/log/jail_console_${name}.log"; ④

  # PERMISSIONS
  allow.raw_sockets; ⑤
  exec.clean; ⑥
  mount.devfs; ⑦

  # HOSTNAME/PATH
  host.hostname = "${name}"; ⑧
  path = "/usr/local/jails/containers/${name}"; ⑨

  # NETWORK
  ip4.addr = 192.168.1.151; ⑩
  ip6.addr = ::ffff:c0a8:197 ⑪
  interface = em0; ⑫
```

```
}
```

① `jailname` - Name of the jail.

② `exec.start` - Command(s) to run in the jail environment when a jail is created. A typical command to run is "/bin/sh /etc/rc".

③ `exec.stop` - Command(s) to run in the jail environment before a jail is removed. A typical command to run is "/bin/sh /etc/rc.shutdown".

④ `exec.consolelog` - A file to direct command output (stdout and stderr) to.

⑤ `allow.raw_sockets` - Allow creating raw sockets inside the jail. Setting this parameter allows utilities like ping(8) and traceroute(8) to operate inside the jail.

⑥ `exec.clean` - Run commands in a clean environment.

⑦ `mount.devfs` - Mount a devfs(5) filesystem on the chrooted /dev directory, and apply the ruleset in the devfs_ruleset parameter to restrict the devices visible inside the jail.

⑧ `host.hostname` - The hostname of the jail.

⑨ `path` - The directory which is to be the root of the jail. Any commands that are run inside the jail, either by jail or from jexec(8), are run from this directory.

⑩ `ip4.addr` - IPv4 address. There are two configuration possibilities for IPv4. The first is to establish an IP or a list of IPs as has been done in the example. The other is to use `ip4` instead and set the `inherit` value to inherit the host's IP address.

⑪ `ip6.addr` - IPv6 address. There are two configuration possibilities for IPv6. The first is to establish an IP or a list of IPs as has been done in the example. The other is to use `ip6` instead and set the `inherit` value to inherit the host's IP address.

⑫ `interface` - A network interface to add the jail's IP addresses. Usually the host interface.

More information about configuration variables can be found in jail(8) and jail.conf(5).

# 17.4. Classic Jail (Thick Jail)

These jails resemble a real FreeBSD system. They can be managed more or less like a normal host system and updated independently.

## 17.4.1. Creating a Classic Jail

In principle, a jail only needs a hostname, a root directory, an IP address, and a userland.

The userland for the jail can be obtained from the official FreeBSD download servers.

Execute the following command to download the userland:

```
# fetch https://download.freebsd.org/ftp/releases/amd64/amd64/15.0-RELEASE/base.txz -o
/usr/local/jails/media/15.0-RELEASE-base.txz
```

Once the download is complete, it will be necessary to extract the contents into the jail directory.

Execute the following commands to extract the userland into the jail's directory:

```
# mkdir -p /usr/local/jails/containers/classic
# tar -xf /usr/local/jails/media/15.0-RELEASE-base.txz -C
/usr/local/jails/containers/classic --unlink
```

With the userland extracted in the jail directory, it will be necessary to copy the timezone and DNS server files:

```
# cp /etc/resolv.conf /usr/local/jails/containers/classic/etc/resolv.conf
# cp /etc/localtime /usr/local/jails/containers/classic/etc/localtime
```

With the files copied, the next thing to do is update to the latest patch level by executing the following command:

```
# freebsd-update -b /usr/local/jails/containers/classic/ fetch install
```

The last step is to configure the jail. It will be necessary to add an entry to the configuration file /etc/jail.conf or in jail.conf.d with the parameters of the jail.

An example would be the following:

```
classic {
  # STARTUP/LOGGING
  exec.start = "/bin/sh /etc/rc";
  exec.stop = "/bin/sh /etc/rc.shutdown";
  exec.consolelog = "/var/log/jail_console_${name}.log";

  # PERMISSIONS
  allow.raw_sockets;
  exec.clean;
  mount.devfs;

  # HOSTNAME/PATH
  host.hostname = "${name}";
  path = "/usr/local/jails/containers/${name}";

  # NETWORK
  ip4.addr = 192.168.1.151;
  interface = em0;
}
```

Execute the following command to start the jail:

```
# service jail start classic
```

More information on how to manage jails can be found in the section Jail Management.

## 17.5. Thin Jails

Although Thin Jails use the same technology as Thick Jails, the creation procedure is different. Thin jails can be created using OpenZFS snapshots or using templates and NullFS. The use of OpenZFS snapshots and templates using NullFS have certain advantages over classic jails, such as being able to create them faster from snapshots or being able to update multiple jails using NullFS.

### 17.5.1. Creating a Thin Jail Using OpenZFS Snapshots

Due to the good integration between FreeBSD and OpenZFS it is very easy to create new Thin Jails using OpenZFS Snapshots.

To create a Thin Jail using OpenZFS Snapshots the first step is to create the jail directory tree by following the instructions in Setting up the Jail Directory Tree.

Next, create a template. Templates will only be used to create new jails. For this reason they are created in "read-only" mode so that jails are created with an immutable base.

To create the dataset for the template, execute the following command:

```
# zfs create -p zroot/jails/templates/15.0-RELEASE
```

Then execute the following command to download the userland:

```
# fetch https://download.freebsd.org/ftp/releases/amd64/amd64/15.0-RELEASE/base.txz -o
/usr/local/jails/media/15.0-RELEASE-base.txz
```

Once the download is complete, it will be necessary to extract the contents in the template directory by executing the following command:

```
# tar -xf /usr/local/jails/media/15.0-RELEASE-base.txz -C
/usr/local/jails/templates/15.0-RELEASE --unlink
```

With the userland extracted in the templates directory, it will be necessary to copy the timezone and DNS server files to the template directory by executing the following command:

```
# cp /etc/resolv.conf /usr/local/jails/templates/15.0-RELEASE/etc/resolv.conf
# cp /etc/localtime /usr/local/jails/templates/15.0-RELEASE/etc/localtime
```

The next thing to do is update to the latest patch level by executing the following command:

```
# freebsd-update -b /usr/local/jails/templates/15.0-RELEASE/ fetch install
```

Once the update is finished, the template is ready.

To create an OpenZFS Snapshot from the template, execute the following command:

```
# zfs snapshot zroot/jails/templates/15.0-RELEASE@base
```

Once the OpenZFS Snapshot has been created, infinite jails can be created using the OpenZFS clone function.

To create a Thin Jail named `thinjail`, execute the following command:

```
# zfs clone zroot/jails/templates/15.0-RELEASE@base zroot/jails/containers/thinjail
```

The last step is to configure the jail. It will be necessary to add an entry to the configuration file /etc/jail.conf or in jail.conf.d with the parameters of the jail.

An example would be the following:

```
thinjail {
  # STARTUP/LOGGING
  exec.start = "/bin/sh /etc/rc";
  exec.stop = "/bin/sh /etc/rc.shutdown";
  exec.consolelog = "/var/log/jail_console_${name}.log";

  # PERMISSIONS
  allow.raw_sockets;
  exec.clean;
  mount.devfs;

  # HOSTNAME/PATH
  host.hostname = "${name}";
  path = "/usr/local/jails/containers/${name}";

  # NETWORK
  ip4 = inherit;
  interface = em0;
}
```

Execute the following command to start the jail:

```
# service jail start thinjail
```

More information on how to manage jails can be found in the section Jail Management.

### 17.5.2. Creating a Thin Jail Using NullFS

A jail can be created with reduced duplication of system files by using the Thin Jail technique and using NullFS to selectively share specific directories from the host system into the jail.

The first step is to create the dataset to save the template, execute the following command if using OpenZFS:

```
# zfs create -p zroot/jails/templates/15.0-RELEASE-base
```

Or this one if using UFS:

```
# mkdir /usr/local/jails/templates/15.0-RELEASE-base
```

Then execute the following command to download the userland:

```
# fetch https://download.freebsd.org/ftp/releases/amd64/amd64/15.0-RELEASE/base.txz -o
/usr/local/jails/media/15.0-RELEASE-base.txz
```

Once the download is complete, it will be necessary to extract the contents in the template directory by executing the following command:

```
# tar -xf /usr/local/jails/media/15.0-RELEASE-base.txz -C
/usr/local/jails/templates/15.0-RELEASE-base --unlink
```

Once the userland is extracted in the templates directory, it will be necessary to copy the timezone and DNS server files to the template directory by executing the following command:

```
# cp /etc/resolv.conf /usr/local/jails/templates/15.0-RELEASE-base/etc/resolv.conf
# cp /etc/localtime /usr/local/jails/templates/15.0-RELEASE-base/etc/localtime
```

With the files moved to the template, the next thing to do is update to the latest patch level by executing the following command:

```
# freebsd-update -b /usr/local/jails/templates/15.0-RELEASE-base/ fetch install
```

In addition to the base template, it is also necessary to create a directory where the `skeleton` will be located. Some directories will be copied from the template to the `skeleton`.

Execute the following command to create the dataset for the `skeleton` in case of using OpenZFS:

```
# zfs create -p zroot/jails/templates/15.0-RELEASE-skeleton
```

Or this one in case of using UFS:

```
# mkdir /usr/local/jails/templates/15.0-RELEASE-skeleton
```

Then create the `skeleton` directories. The `skeleton` directories will hold the local directories of the jails.

Execute the following commands to create the directories:

```
# mkdir -p /usr/local/jails/templates/15.0-RELEASE-skeleton/home
# mkdir -p /usr/local/jails/templates/15.0-RELEASE-skeleton/usr
# mv /usr/local/jails/templates/15.0-RELEASE-base/etc /usr/local/jails/templates/15.0-
RELEASE-skeleton/etc
# mv /usr/local/jails/templates/15.0-RELEASE-base/usr/local
/usr/local/jails/templates/15.0-RELEASE-skeleton/usr/local
# mv /usr/local/jails/templates/15.0-RELEASE-base/tmp /usr/local/jails/templates/15.0-
RELEASE-skeleton/tmp
# mv /usr/local/jails/templates/15.0-RELEASE-base/var /usr/local/jails/templates/15.0-
RELEASE-skeleton/var
# mv /usr/local/jails/templates/15.0-RELEASE-base/root
/usr/local/jails/templates/15.0-RELEASE-skeleton/root
```

The next step is to create the symlinks to the `skeleton` by executing the following commands:

```
# cd /usr/local/jails/templates/15.0-RELEASE-base/
# mkdir skeleton
# ln -s skeleton/etc etc
# ln -s skeleton/home home
# ln -s skeleton/root root
# ln -s ../skeleton/usr/local usr/local
# ln -s skeleton/tmp tmp
# ln -s skeleton/var var
```

With the `skeleton` ready, it will be necessary to copy the data to the jail directory.

In case of using OpenZFS, OpenZFS snapshots can be used to easily create as many jails as necessary by executing the following commands:

```
# zfs snapshot zroot/jails/templates/15.0-RELEASE-skeleton@base
# zfs clone zroot/jails/templates/15.0-RELEASE-skeleton@base
zroot/jails/containers/thinjail
```

In case of using UFS the cp(1) program can be used by executing the following command:

```
# cp -R /usr/local/jails/templates/15.0-RELEASE-skeleton
```

```
/usr/local/jails/containers/thinjail
```

Then create the directory in which the base template and the skeleton will be mounted:

```
# mkdir -p /usr/local/jails/thinjail-nullfs-base
```

Add a jail entry in /etc/jail.conf or a file in jail.conf.d as follows:

```
thinjail {
  # STARTUP/LOGGING
  exec.start = "/bin/sh /etc/rc";
  exec.stop = "/bin/sh /etc/rc.shutdown";
  exec.consolelog = "/var/log/jail_console_${name}.log";

  # PERMISSIONS
  allow.raw_sockets;
  exec.clean;
  mount.devfs;

  # HOSTNAME/PATH
  host.hostname = "${name}";
  path = "/usr/local/jails/${name}-nullfs-base";

  # NETWORK
  ip4.addr = 192.168.1.153;
  interface = em0;

  # MOUNT
  mount.fstab = "/usr/local/jails/${name}-nullfs-base.fstab";
}
```

Then the create the /usr/local/jails/thinjail-nullfs-base.fstab file as follows:

```
/usr/local/jails/templates/15.0-RELEASE-base  /usr/local/jails/thinjail-nullfs-base/
nullfs   ro          0 0
/usr/local/jails/containers/thinjail     /usr/local/jails/thinjail-nullfs-
base/skeleton nullfs  rw  0 0
```

Execute the following command to start the jail:

```
# service jail start thinjail
```

### 17.5.3. Creating a VNET Jail

FreeBSD VNET Jails have their own distinct networking stack, including interfaces, IP addresses,

routing tables, and firewall rules.

The first step to create a VNET jail is to create the bridge(4) by executing the following command:

```
# ifconfig bridge create
```

The output should be similar to the following:

```
bridge0
```

With the `bridge` created, it will be necessary to attach it to the `em0` interface and bring both of them up by executing the following commands:

```
# ifconfig bridge0 addm em0 up
# ifconfig em0 up
```

To make this setting persist across reboots, add the following lines to /etc/rc.conf:

```
defaultrouter="192.168.1.1"
cloned_interfaces="bridge0"
ifconfig_bridge0="inet 192.168.1.150/24 addm em0 up"
ifconfig_em0="up"
```

For more information on bridging, see Network Bridging.

The next step is to create the jail as indicated above.

Either the Classic Jail (Thick Jail) procedure and the Thin Jails procedure can be used. The only thing that will change is the configuration in the /etc/jail.conf file.

The path /usr/local/jails/containers/vnet will be used as an example for the created jail.

The following is an example configuration for a VNET jail:

```
vnet {
  # STARTUP/LOGGING
  exec.consolelog = "/var/log/jail_console_${name}.log";

  # PERMISSIONS
  allow.raw_sockets;
  exec.clean;
  mount.devfs;
  devfs_ruleset = 5;

  # PATH/HOSTNAME
  path = "/usr/local/jails/containers/${name}";
```

```
  host.hostname = "${name}";

  # VNET/VIMAGE
  vnet;
  vnet.interface = "${epair}b";

  # NETWORKS/INTERFACES
  $id = "154"; ①
  $ip = "192.168.1.${id}/24";
  $gateway = "192.168.1.1";
  $bridge = "bridge0"; ②
  $epair = "epair${id}";

  # ADD TO bridge INTERFACE
  exec.prestart  = "/sbin/ifconfig ${epair} create up";
  exec.prestart += "/sbin/ifconfig ${epair}a up descr jail:${name}";
  exec.prestart += "/sbin/ifconfig ${bridge} addm ${epair}a up";
  exec.start    += "/sbin/ifconfig ${epair}b ${ip} up";
  exec.start    += "/sbin/route add default ${gateway}";
  exec.start    += "/bin/sh /etc/rc";
  exec.stop = "/bin/sh /etc/rc.shutdown";
  exec.poststop = "/sbin/ifconfig ${bridge} deletem ${epair}a";
  exec.poststop += "/sbin/ifconfig ${epair}a destroy";
}
```

① Represents the IP of the Jail, it must be **unique**.

② Refers to the bridge created previously.

### 17.5.4. Creating a Linux Jail

FreeBSD can run Linux inside a jail using Linux Binary Compatibility and debootstrap(8). Jails do not have a kernel. They run on the host's kernel. Therefore it is necessary to enable Linux Binary Compatibility in the host system.

To enable the Linux ABI at boot time, execute the following command:

```
# sysrc linux_enable="YES"
```

Once enabled, it can be started without rebooting by executing the following command:

```
# service linux start
```

The next step will be to create a jail as indicated above, for example in Creating a Thin Jail Using OpenZFS Snapshots, but **without** performing the configuration. FreeBSD Linux jails require a specific configuration that will be detailed below.

Once the jail has been created as explained above, execute the following command to perform required configuration for the jail and start it:

```
# jail -cm \
    name=ubuntu \
    host.hostname="ubuntu.example.com" \
    path="/usr/local/jails/ubuntu" \
    interface="em0" \
    ip4.addr="192.168.1.150" \
    exec.start="/bin/sh /etc/rc" \
    exec.stop="/bin/sh /etc/rc.shutdown" \
    mount.devfs \
    devfs_ruleset=4 \
    allow.mount \
    allow.mount.devfs \
    allow.mount.fdescfs \
    allow.mount.procfs \
    allow.mount.linprocfs \
    allow.mount.linsysfs \
    allow.mount.tmpfs \
    enforce_statfs=1
```

To access the jail, it will be necessary to install sysutils/debootstrap.

Execute the following command to access the FreeBSD Linux jail:

```
# jexec -u root ubuntu
```

Inside the jail, execute the following commands to install sysutils/debootstrap and prepare the Ubuntu environment:

```
# pkg install debootstrap
# debootstrap jammy /compat/ubuntu
```

When the process has finished and the message `Base system installed successfully` is displayed on the console, it will be necessary to stop the jail from the host system by executing the following command:

```
# service jail onestop ubuntu
```

Then add an entry in /etc/jail.conf for the Linux jail:

```
ubuntu {
  # STARTUP/LOGGING
  exec.start = "/bin/sh /etc/rc";
  exec.stop = "/bin/sh /etc/rc.shutdown";
  exec.consolelog = "/var/log/jail_console_${name}.log";
```

```
  # PERMISSIONS
  allow.raw_sockets;
  exec.clean;
  mount.devfs;
  devfs_ruleset = 4;

  # HOSTNAME/PATH
  host.hostname = "${name}";
  path = "/usr/local/jails/containers/${name}";

  # NETWORK
  ip4.addr = 192.168.1.155;
  interface = em0;

  # MOUNT
  mount += "devfs     $path/compat/ubuntu/dev     devfs     rw  0 0";
  mount += "tmpfs     $path/compat/ubuntu/dev/shm tmpfs     rw,size=1g,mode=1777  0
0";
  mount += "fdescfs   $path/compat/ubuntu/dev/fd  fdescfs   rw,linrdlnk 0 0";
  mount += "linprocfs $path/compat/ubuntu/proc    linprocfs rw  0 0";
  mount += "linsysfs  $path/compat/ubuntu/sys     linsysfs  rw  0 0";
  mount += "/tmp      $path/compat/ubuntu/tmp     nullfs    rw  0 0";
  mount += "/home     $path/compat/ubuntu/home    nullfs    rw  0 0";
}
```

Then the jail can be started as usual with the following command:

```
# service jail start ubuntu
```

The Ubuntu environment can be accessed using the following command:

```
# jexec ubuntu chroot /compat/ubuntu /bin/bash
```

More information can be found in the chapter Linux Binary Compatibility.

### 17.5.5. Configuring Service Jails

A service jail is configured completely via /etc/rc.conf or sysrc(8). The base system services are service jails ready. They contain a config line which enables networking or lift other restrictions of jails. Base system services which do not make sense to run inside jails are configured to not be started as a service jail, even if enabled in /etc/rc.conf. Some examples of such a service are services which want to mount or unmount something in the start of stop method, or only configure something like a route, or firewall, or the like.

Third party services may or may not be service jails ready. To check if a service is service jail ready, the following command can be used:

```
# grep _svcj_options /path/to/rc.d/servicename
```

If there is no output, the service is not service jail ready, or does not need any additional privileges like for example, network access.

If the service is not service jail ready, and needs network access, it can be made ready by adding the necessary config to /etc/rc.conf:

```
# sysrc servicename_svcj_options=net_basic
```

For all possible `_svcj_options` see the rc.conf(5) man-page.

To enable a service jail for a given service, the service needs to be stopped and the `servicename_svcj` variable needs to be set to YES. To put syslogd(8) into a service jail, use the following sequence of commands:

```
# service syslogd stop
# sysrc syslogd_svcj=YES
# service syslogd start
```

If the `servicename_svcj` variable is changed, the service needs to be stopped before it is changed. If it is not stopped, the rc framework will not detect the correct state of the service and will not be able to do what is requested.

Service jails are managed only via rc.conf(5)/sysrc(8) and the service(8) command. The jail utilities, like jls(8) as described in Jail Management can be used to investigate the operation, but the jail(8) command is not supposed to be used to manage them.

# 17.6. Jail Management

Once the jail is created, there are a number of operations that can be performed, like starting, rebooting or deleting the jail, installing software in it, etc. In this section the different actions that can be done with jails from the host will be described.

## 17.6.1. List Running Jails

To list the jails that are running on the host system, the command jls(8) can be used:

```
# jls
```

The output should be similar to the following:

```
   JID  IP Address      Hostname                      Path
     1  192.168.250.70  classic
```

```
/usr/local/jails/containers/classic
```

jls(8) supports the `--libxo` argument, which through the libxo(3) library allows other types of formats to be displayed, such as `JSON`, `HTML`, etc.

For example, execute the following command to get the `JSON` output:

```
# jls --libxo=json
```

The output should be similar to the following:

```
{"__version": "2", "jail-information": {"jail":
[{"jid":1,"ipv4":"192.168.250.70","hostname":"classic","path":"/usr/local/jails/contai
ners/classic"}]}}
```

### 17.6.2. Start, Restart, and Stop a Jail

service(8) is used to start, reboot, or stop a jail on the host.

For example, to start a jail, run the following command:

```
# service jail start jailname
```

Change the `start` argument to `restart` or `stop` to perform other actions on the jail.

### 17.6.3. Destroy a Jail

Destroying a jail is not as simple as stopping the jail using service(8) and removing the jail directory and /etc/jail.conf entry.

FreeBSD takes system security very seriously. For this reason there are certain files that not even the root user can delete. This functionality is known as File Flags.

The first step is to stop the desired jail executing the following command:

```
# service jail stop jailname
```

The second step is to remove these flags with chflags(1) by executing the following command, in which `classic` is the name of the jail to remove:

```
# chflags -R 0 /usr/local/jails/containers/classic
```

The third step is to delete the directory where the jail was:

```
# rm -rf /usr/local/jails/containers/classic
```

Finally, it will be necessary to remove the jail entry in /etc/jail.conf or in jail.conf.d.

### 17.6.4. Handle Packages in a Jail

The pkg(8) tool supports the `-j` argument in order to handle packages installed inside the jail.

For example, to install www/nginx-lite in the jail, the next command can be executed **from the host**:

```
# pkg -j classic install nginx-lite
```

For more information on working with packages in FreeBSD, see Installing Applications: Packages and Ports.

### 17.6.5. Access a Jail

While it has been stated above that it is best to manage jails from the host system, a jail can be entered with jexec(8).

The jail can be entered by running jexec(8) from the host:

```
# jexec -u root jailname
```

When gaining access to the jail, the message configured in motd(5) will be displayed.

### 17.6.6. Execute Commands in a Jail

To execute a command from the host system in a jail the jexec(8) can be used.

For example, to stop a service that is running inside a jail, the command will be executed:

```
# jexec -l jailname service nginx stop
```

## 17.7. Jail Upgrading

Upgrading FreeBSD Jails ensures that the isolated environments remain secure, up-to-date, and in line with the latest features and improvements available in the FreeBSD ecosystem.

### 17.7.1. Upgrading a Classic Jail or a Thin Jail using OpenZFS Snapshots

Jails **must be updated from the host** operating system. The default behavior in FreeBSD is to disallow the use of chflags(1) in a jail. This will prevent the update of some files so updating from within the jail will fail.

To update the jail to the latest patch release of the version of FreeBSD it is running, execute the following commands on the host:

```
# freebsd-update -j classic fetch install
# service jail restart classic
```

To upgrade the jail to a new major or minor version, first upgrade the host system as described in Performing Major and Minor Version Upgrades. Once the host has been upgraded and rebooted, the jail can then be upgraded.



In case of upgrade from one version to another, it is easier to create a new jail than to upgrade completely.

For example to upgrade from 13.1-RELEASE to 13.2-RELEASE, execute the following commands on the host:

```
# freebsd-update -j classic -r 13.2-RELEASE upgrade
# freebsd-update -j classic install
# service jail restart classic
# freebsd-update -j classic install
# service jail restart classic
```



It is necessary to execute the `install` step two times. The first one upgrades the kernel, and the second one upgrades the rest of the components.

Then, if it was a major version upgrade, reinstall all installed packages and restart the jail again. This is required because the ABI version changes when upgrading between major versions of FreeBSD.

From the host:

```
# pkg -j jailname upgrade -f
# service jail restart jailname
```

### 17.7.2. Upgrading a Thin Jail Using NullFS

Since Thin Jails that use NullFS share the majority of system directories, they are very easy to update. It is enough to update the template. This allows updating multiple jails at the same time.

To update the template to the latest patch release of the version of FreeBSD it is running, execute the following commands on the host:

```
# freebsd-update -b /usr/local/jails/templates/13.1-RELEASE-base/ fetch install
# service jail restart
```

To upgrade the template to a new major or minor version, first upgrade the host system as described in Performing Major and Minor Version Upgrades. Once the host has been upgraded and rebooted, the template can then be upgraded.

For example, to upgrade from 13.1-RELEASE to 13.2-RELEASE, execute the following commands on the host:

```
# freebsd-update -b /usr/local/jails/templates/13.1-RELEASE-base/ -r 13.2-RELEASE
upgrade
# freebsd-update -b /usr/local/jails/templates/13.1-RELEASE-base/ install
# service jail restart
# freebsd-update -b /usr/local/jails/templates/13.1-RELEASE-base/ install
# service jail restart
```

## 17.8. Jail Resource Limits

Controlling the resources that a jail uses from the host system is a task to be taken into account by the system administrator.

Use rctl(8) to manage the resources that a jail can use from the host system.



The `kern.racct.enable` tunable must be enabled at /boot/loader.conf.

The syntax to limit the resources of a jail is as follows:

```
rctl -a jail:<jailname>:resource:action=amount/percentage
```

For example, to limit the maximum RAM that a jail can access, run the following command:

```
# rctl -a jail:classic:memoryuse:deny=2G
```

To make the limitation persistent across reboots of the host system, it will be necessary to add the rule to the /etc/rctl.conf file as follows:

```
jail:classic:memoryuse:deny=2G/jail
```

More information on resource limits can be found in the security chapter in the Resource Limits section.

## 17.9. Jail Managers and Containers

As previously explained, each type of FreeBSD Jail can be created and configured manually, but FreeBSD also has third-party utilities to make configuration and administration easier.

Below is an incomplete list of the different FreeBSD Jail managers:

*Table 31. Jail Managers*

| Name | License | Package | Documentation |
|---|---|---|---|
| BastilleBSD | BSD-3 | sysutils/bastille | Documentation |
| pot | BSD-3 | sysutils/pot | Documentation |
| cbsd | BSD-2 | sysutils/cbsd | Documentation |
| AppJail | BSD-3 | sysutils/appjail, for devel sysutils/appjail-devel | Documentation |
| iocage | BSD-2 | sysutils/iocage | Documentation |
| ezjail | Beer Ware | sysutils/ezjail | Documentation |

# Chapter 18. Mandatory Access Control

## 18.1. Synopsis

FreeBSD supports security extensions based on the POSIX®.1e draft. These security mechanisms include file system Access Control Lists (“Access Control Lists”) and Mandatory Access Control (MAC). MAC allows access control modules to be loaded in order to implement security policies. Some modules provide protections for a narrow subset of the system, hardening a particular service. Others provide comprehensive labeled security across all subjects and objects. The mandatory part of the definition indicates that enforcement of controls is performed by administrators and the operating system. This is in contrast to the default security mechanism of Discretionary Access Control (DAC) where enforcement is left to the discretion of users.

This chapter focuses on the MAC framework and the set of pluggable security policy modules FreeBSD provides for enabling various security mechanisms.

Read this chapter to learn:

- The terminology associated with the MAC framework.
- The capabilities of MAC security policy modules as well as the difference between a labeled and non-labeled policy.
- The considerations to take into account before configuring a system to use the MAC framework.
- Which MAC security policy modules are included in FreeBSD and how to configure them.
- How to implement a more secure environment using the MAC framework.
- How to test the MAC configuration to ensure the framework has been properly implemented.

Before reading this chapter:

- Understand UNIX® and FreeBSD basics (FreeBSD Basics).
- Have some familiarity with security and how it pertains to FreeBSD (Security).



Improper MAC configuration may cause loss of system access, aggravation of users, or inability to access the features provided by Xorg. More importantly, MAC should not be relied upon to completely secure a system. The MAC framework only augments an existing security policy. Without sound security practices and regular security checks, the system will never be completely secure.

The examples contained within this chapter are for demonstration purposes and the example settings should *not* be implemented on a production system. Implementing any security policy takes a good deal of understanding, proper design, and thorough testing.

While this chapter covers a broad range of security issues relating to the MAC framework, the development of new MAC security policy modules will not be covered. A number of security policy modules included with the MAC framework have specific characteristics which are provided for both testing and new module development. Refer to mac_test(4), mac_stub(4) and mac_none(4) for

more information on these security policy modules and the various mechanisms they provide.

## 18.2. Key Terms

The following key terms are used when referring to the MAC framework:

- *compartment*: a set of programs and data to be partitioned or separated, where users are given explicit access to specific component of a system. A compartment represents a grouping, such as a work group, department, project, or topic. Compartments make it possible to implement a need-to-know-basis security policy.
- *integrity*: the level of trust which can be placed on data. As the integrity of the data is elevated, so does the ability to trust that data.
- *level*: the increased or decreased setting of a security attribute. As the level increases, its security is considered to elevate as well.
- *label*: a security attribute which can be applied to files, directories, or other items in the system. It could be considered a confidentiality stamp. When a label is placed on a file, it describes the security properties of that file and will only permit access by files, users, and resources with a similar security setting. The meaning and interpretation of label values depends on the policy configuration. Some policies treat a label as representing the integrity or secrecy of an object while other policies might use labels to hold rules for access.
- *multilabel*: this property is a file system option which can be set in single-user mode using tunefs(8), during boot using fstab(5), or during the creation of a new file system. This option permits an administrator to apply different MAC labels on different objects. This option only applies to security policy modules which support labeling.
- *single label*: a policy where the entire file system uses one label to enforce access control over the flow of data. Whenever `multilabel` is not set, all files will conform to the same label setting.
- *object*: an entity through which information flows under the direction of a *subject*. This includes directories, files, fields, screens, keyboards, memory, magnetic storage, printers or any other data storage or moving device. An object is a data container or a system resource. Access to an object effectively means access to its data.
- *subject*: any active entity that causes information to flow between *objects* such as a user, user process, or system process. On FreeBSD, this is almost always a thread acting in a process on behalf of a user.
- *policy*: a collection of rules which defines how objectives are to be achieved. A policy usually documents how certain items are to be handled. This chapter considers a policy to be a collection of rules which controls the flow of data and information and defines who has access to that data and information.
- *high-watermark*: this type of policy permits the raising of security levels for the purpose of accessing higher level information. In most cases, the original level is restored after the process is complete. Currently, the FreeBSD MAC framework does not include this type of policy.
- *low-watermark*: this type of policy permits lowering security levels for the purpose of accessing information which is less secure. In most cases, the original security level of the user is restored after the process is complete. The only security policy module in FreeBSD to use this is mac_lomac(4).

- *sensitivity*: usually used when discussing Multilevel Security (MLS). A sensitivity level describes how important or secret the data should be. As the sensitivity level increases, so does the importance of the secrecy, or confidentiality, of the data.

## 18.3. Understanding MAC Labels

A MAC label is a security attribute which may be applied to subjects and objects throughout the system. When setting a label, the administrator must understand its implications in order to prevent unexpected or undesired behavior of the system. The attributes available on an object depend on the loaded policy module, as policy modules interpret their attributes in different ways.

The security label on an object is used as a part of a security access control decision by a policy. With some policies, the label contains all of the information necessary to make a decision. In other policies, the labels may be processed as part of a larger rule set.

There are two types of label policies: single label and multi label. By default, the system will use single label. The administrator should be aware of the pros and cons of each in order to implement policies which meet the requirements of the system's security model.

A single label security policy only permits one label to be used for every subject or object. Since a single label policy enforces one set of access permissions across the entire system, it provides lower administration overhead, but decreases the flexibility of policies which support labeling. However, in many environments, a single label policy may be all that is required.

A single label policy is somewhat similar to DAC as `root` configures the policies so that users are placed in the appropriate categories and access levels. A notable difference is that many policy modules can also restrict `root`. Basic control over objects will then be released to the group, but `root` may revoke or modify the settings at any time.

When appropriate, a multi label policy can be set on a UFS file system by passing `multilabel` to tunefs(8). A multi label policy permits each subject or object to have its own independent MAC label. The decision to use a multi label or single label policy is only required for policies which implement the labeling feature, such as `biba`, `lomac`, and `mls`. Some policies, such as `seeotheruids`, `portacl` and `partition`, do not use labels at all.

Using a multi label policy on a partition and establishing a multi label security model can increase administrative overhead as everything in that file system has a label. This includes directories, files, and even device nodes.

The following command will set `multilabel` on the specified UFS file system. This may only be done in single-user mode and is not a requirement for the swap file system:

```
# tunefs -l enable /
```



Some users have experienced problems with setting the `multilabel` flag on the root partition. If this is the case, please review Troubleshooting the MAC Framework.

Since the multi label policy is set on a per-file system basis, a multi label policy may not be needed

if the file system layout is well designed. Consider an example security MAC model for a FreeBSD web server. This machine uses the single label, `biba/high`, for everything in the default file systems. If the web server needs to run at `biba/low` to prevent write up capabilities, it could be installed to a separate UFS /usr/local file system set at `biba/low`.

### 18.3.1. Label Configuration

Virtually all aspects of label policy module configuration will be performed using the base system utilities. These commands provide a simple interface for object or subject configuration or the manipulation and verification of the configuration.

All configuration may be done using `setfmac`, which is used to set MAC labels on system objects, and `setpmac`, which is used to set the labels on system subjects. For example, to set the `biba` MAC label to `high` on test:

```
# setfmac biba/high test
```

If the configuration is successful, the prompt will be returned without error. A common error is `Permission denied` which usually occurs when the label is being set or modified on a restricted object. Other conditions may produce different failures. For instance, the file may not be owned by the user attempting to relabel the object, the object may not exist, or the object may be read-only. A mandatory policy will not allow the process to relabel the file, maybe because of a property of the file, a property of the process, or a property of the proposed new label value. For example, if a user running at low integrity tries to change the label of a high integrity file, or a user running at low integrity tries to change the label of a low integrity file to a high integrity label, these operations will fail.

The system administrator may use `setpmac` to override the policy module's settings by assigning a different label to the invoked process:

```
# setfmac biba/high test
Permission denied
# setpmac biba/low setfmac biba/high test
# getfmac test
test: biba/high
```

For currently running processes, such as sendmail, `getpmac` is usually used instead. This command takes a process ID (PID) in place of a command name. If users attempt to manipulate a file not in their access, subject to the rules of the loaded policy modules, the `Operation not permitted` error will be displayed.

### 18.3.2. Predefined Labels

A few FreeBSD policy modules which support the labeling feature offer three predefined labels: `low`, `equal`, and `high`, where:

- `low` is considered the lowest label setting an object or subject may have. Setting this on objects

or subjects blocks their access to objects or subjects marked high.

- `equal` sets the subject or object to be disabled or unaffected and should only be placed on objects considered to be exempt from the policy.
- `high` grants an object or subject the highest setting available in the Biba and MLS policy modules.

Such policy modules include mac_biba(4), mac_mls(4) and mac_lomac(4). Each of the predefined labels establishes a different information flow directive. Refer to the manual page of the module to determine the traits of the generic label configurations.

### 18.3.3. Numeric Labels

The Biba and MLS policy modules support a numeric label which may be set to indicate the precise level of hierarchical control. This numeric level is used to partition or sort information into different groups of classification, only permitting access to that group or a higher group level. For example:

```
biba/10:2+3+6(5:2+3-20:2+3+4+5+6)
```

may be interpreted as "Biba Policy Label/Grade 10:Compartments 2, 3 and 6: (grade 5 ...")

In this example, the first grade would be considered the effective grade with effective compartments, the second grade is the low grade, and the last one is the high grade. In most configurations, such fine-grained settings are not needed as they are considered to be advanced configurations.

System objects only have a current grade and compartment. System subjects reflect the range of available rights in the system, and network interfaces, where they are used for access control.

The grade and compartments in a subject and object pair are used to construct a relationship known as *dominance*, in which a subject dominates an object, the object dominates the subject, neither dominates the other, or both dominate each other. The "both dominate" case occurs when the two labels are equal. Due to the information flow nature of Biba, a user has rights to a set of compartments that might correspond to projects, but objects also have a set of compartments. Users may have to subset their rights using `su` or `setpmac` in order to access objects in a compartment from which they are not restricted.

### 18.3.4. User Labels

Users are required to have labels so that their files and processes properly interact with the security policy defined on the system. This is configured in /etc/login.conf using login classes. Every policy module that uses labels will implement the user class setting.

To set the user class default label which will be enforced by MAC, add a `label` entry. An example `label` entry containing every policy module is displayed below. Note that in a real configuration, the administrator would never enable every policy module. It is recommended that the rest of this chapter be reviewed before any configuration is implemented.

```
default:\
    :welcome=/etc/motd:\
    :setenv=MAIL=/var/mail/$,BLOCKSIZE=K:\
    :path=~/bin:/sbin:/bin:/usr/sbin:/usr/bin:/usr/local/sbin:/usr/local/bin:\
    :manpath=/usr/share/man /usr/local/man:\
    :nologin=/usr/sbin/nologin:\
    :cputime=1h30m:\
    :datasize=8M:\
    :vmemoryuse=100M:\
    :stacksize=2M:\
    :memorylocked=4M:\
    :memoryuse=8M:\
    :filesize=8M:\
    :coredumpsize=8M:\
    :openfiles=24:\
    :maxproc=32:\
    :priority=0:\
    :requirehome:\
    :passwordtime=91d:\
    :umask=022:\
    :ignoretime@:\
    :label=partition/13,mls/5,biba/10(5-15),lomac/10[2]:
```

While users can not modify the default value, they may change their label after they login, subject to the constraints of the policy. The example above tells the Biba policy that a process's minimum integrity is `5`, its maximum is `15`, and the default effective label is `10`. The process will run at `10` until it chooses to change label, perhaps due to the user using `setpmac`, which will be constrained by Biba to the configured range.

After any change to login.conf, the login class capability database must be rebuilt using `cap_mkdb`.

Many sites have a large number of users requiring several different user classes. In depth planning is required as this can become difficult to manage.

### 18.3.5. Network Interface Labels

Labels may be set on network interfaces to help control the flow of data across the network. Policies using network interface labels function in the same way that policies function with respect to objects. Users at high settings in Biba, for example, will not be permitted to access network interfaces with a label of `low`.

When setting the MAC label on network interfaces, `maclabel` may be passed to `ifconfig`:

```
# ifconfig bge0 maclabel biba/equal
```

This example will set the MAC label of `biba/equal` on the `bge0` interface. When using a setting similar to `biba/high(low-high)`, the entire label should be quoted to prevent an error from being returned.

Each policy module which supports labeling has a tunable which may be used to disable the MAC label on network interfaces. Setting the label to `equal` will have a similar effect. Review the output of `sysctl`, the policy manual pages, and the information in the rest of this chapter for more information on those tunables.

## 18.4. Planning the Security Configuration

Before implementing any MAC policies, a planning phase is recommended. During the planning stages, an administrator should consider the implementation requirements and goals, such as:

- How to classify information and resources available on the target systems.
- Which information or resources to restrict access to along with the type of restrictions that should be applied.
- Which MAC modules will be required to achieve this goal.

A trial run of the trusted system and its configuration should occur *before* a MAC implementation is used on production systems. Since different environments have different needs and requirements, establishing a complete security profile will decrease the need of changes once the system goes live.

Consider how the MAC framework augments the security of the system as a whole. The various security policy modules provided by the MAC framework could be used to protect the network and file systems or to block users from accessing certain ports and sockets. Perhaps the best use of the policy modules is to load several security policy modules at a time in order to provide a MLS environment. This approach differs from a hardening policy, which typically hardens elements of a system which are used only for specific purposes. The downside to MLS is increased administrative overhead.

The overhead is minimal when compared to the lasting effect of a framework which provides the ability to pick and choose which policies are required for a specific configuration and which keeps performance overhead down. The reduction of support for unneeded policies can increase the overall performance of the system as well as offer flexibility of choice. A good implementation would consider the overall security requirements and effectively implement the various security policy modules offered by the framework.

A system utilizing MAC guarantees that a user will not be permitted to change security attributes at will. All user utilities, programs, and scripts must work within the constraints of the access rules provided by the selected security policy modules and control of the MAC access rules is in the hands of the system administrator.

It is the duty of the system administrator to carefully select the correct security policy modules. For an environment that needs to limit access control over the network, the mac_portacl(4), mac_ifoff(4), and mac_biba(4) policy modules make good starting points. For an environment where strict confidentiality of file system objects is required, consider the mac_bsdextended(4) and mac_mls(4) policy modules.

Policy decisions could be made based on network configuration. If only certain users should be permitted access to ssh(1), the mac_portacl(4) policy module is a good choice. In the case of file systems, access to objects might be considered confidential to some users, but not to others. As an

example, a large development team might be broken off into smaller projects where developers in project A might not be permitted to access objects written by developers in project B. Yet both projects might need to access objects created by developers in project C. Using the different security policy modules provided by the MAC framework, users could be divided into these groups and then given access to the appropriate objects.

Each security policy module has a unique way of dealing with the overall security of a system. Module selection should be based on a well thought out security policy which may require revision and reimplementation. Understanding the different security policy modules offered by the MAC framework will help administrators choose the best policies for their situations.

The rest of this chapter covers the available modules, describes their use and configuration, and in some cases, provides insight on applicable situations.



Implementing MAC is much like implementing a firewall since care must be taken to prevent being completely locked out of the system. The ability to revert back to a previous configuration should be considered and the implementation of MAC over a remote connection should be done with extreme caution.

## 18.5. Available MAC Policies

The default FreeBSD kernel includes `options MAC`. This means that every module included with the MAC framework can be loaded with `kldload` as a run-time kernel module. After testing the module, add the module name to /boot/loader.conf so that it will load during boot. Each module also provides a kernel option for those administrators who choose to compile their own custom kernel.

FreeBSD includes a group of policies that will cover most security requirements. Each policy is summarized below. The last three policies support integer settings in place of the three default labels.

### 18.5.1. The MAC See Other UIDs Policy

Module name: mac_seeotheruids.ko

Kernel configuration line: `options MAC_SEEOTHERUIDS`

Boot option: `mac_seeotheruids_load="YES"`

The mac_seeotheruids(4) module extends the `security.bsd.see_other_uids` and `security.bsd.see_other_gids` `sysctl` tunables. This option does not require any labels to be set before configuration and can operate transparently with other modules.

After loading the module, the following `sysctl` tunables may be used to control its features:

- `security.mac.seeotheruids.enabled` enables the module and implements the default settings which deny users the ability to view processes and sockets owned by other users.
- `security.mac.seeotheruids.specificgid_enabled` allows specified groups to be exempt from this policy. To exempt specific groups, use the `security.mac.seeotheruids.specificgid=XXX` `sysctl` tunable, replacing *XXX* with the numeric group ID to be exempted.

- `security.mac.seeotheruids.primarygroup_enabled` is used to exempt specific primary groups from this policy. When using this tunable, `security.mac.seeotheruids.specificgid_enabled` may not be set.

### 18.5.2. The MAC BSD Extended Policy

Module name: mac_bsdextended.ko

Kernel configuration line: `options MAC_BSDEXTENDED`

Boot option: `mac_bsdextended_load="YES"`

The mac_bsdextended(4) module enforces a file system firewall. It provides an extension to the standard file system permissions model, permitting an administrator to create a firewall-like ruleset to protect files, utilities, and directories in the file system hierarchy. When access to a file system object is attempted, the list of rules is iterated until either a matching rule is located or the end is reached. This behavior may be changed using `security.mac.bsdextended.firstmatch_enabled`. Similar to other firewall modules in FreeBSD, a file containing the access control rules can be created and read by the system at boot time using an rc.conf(5) variable.

The rule list may be entered using ugidfw(8) which has a syntax similar to ipfw(8). More tools can be written by using the functions in the libugidfw(3) library.

After the mac_bsdextended(4) module has been loaded, the following command may be used to list the current rule configuration:

```
# ugidfw list
0 slots, 0 rules
```

By default, no rules are defined and everything is completely accessible. To create a rule which blocks all access by users but leaves `root` unaffected:

```
# ugidfw add subject not uid root new object not uid root mode n
```

While this rule is simple to implement, it is a very bad idea as it blocks all users from issuing any commands. A more realistic example blocks `user1` all access, including directory listings, to `user2`'s home directory:

```
# ugidfw set 2 subject uid user1 object uid user2 mode n
# ugidfw set 3 subject uid user1 object gid user2 mode n
```

Instead of `user1`, `not uid user2` could be used in order to enforce the same access restrictions for all users. However, the `root` user is unaffected by these rules.



Extreme caution should be taken when working with this module as incorrect use could block access to certain parts of the file system.

### 18.5.3. The MAC Interface Silencing Policy

Module name: mac_ifoff.ko

Kernel configuration line: `options MAC_IFOFF`

Boot option: `mac_ifoff_load="YES"`

The mac_ifoff(4) module is used to disable network interfaces on the fly and to keep network interfaces from being brought up during system boot. It does not use labels and does not depend on any other MAC modules.

Most of this module's control is performed through these `sysctl` tunables:

- `security.mac.ifoff.lo_enabled` enables or disables all traffic on the loopback, lo(4), interface.
- `security.mac.ifoff.bpfrecv_enabled` enables or disables all traffic on the Berkeley Packet Filter interface, bpf(4).
- `security.mac.ifoff.other_enabled` enables or disables traffic on all other interfaces.

One of the most common uses of mac_ifoff(4) is network monitoring in an environment where network traffic should not be permitted during the boot sequence. Another use would be to write a script which uses an application such as security/aide to automatically block network traffic if it finds new or altered files in protected directories.

### 18.5.4. The MAC Port Access Control List Policy

Module name: mac_portacl.ko

Kernel configuration line: `MAC_PORTACL`

Boot option: `mac_portacl_load="YES"`

The mac_portacl(4) module is used to limit binding to local TCP and UDP ports, making it possible to allow non-`root` users to bind to specified privileged ports below 1024.

Once loaded, this module enables the MAC policy on all sockets. The following tunables are available:

- `security.mac.portacl.enabled` enables or disables the policy completely.
- `security.mac.portacl.port_high` sets the highest port number that mac_portacl(4) protects.
- `security.mac.portacl.suser_exempt`, when set to a non-zero value, exempts the `root` user from this policy.
- `security.mac.portacl.rules` specifies the policy as a text string of the form `rule[,rule,…]`, with as many rules as needed, and where each rule is of the form `idtype:id:protocol:port`. The `idtype` is either `uid` or `gid`. The `protocol` parameter can be `tcp` or `udp`. The `port` parameter is the port number to allow the specified user or group to bind to. Only numeric values can be used for the user ID, group ID, and port parameters.

By default, ports below 1024 can only be used by privileged processes which run as `root`. For

mac_portacl(4) to allow non-privileged processes to bind to ports below 1024, set the following tunables as follows:

```
# sysctl security.mac.portacl.port_high=1023
# sysctl net.inet.ip.portrange.reservedlow=0
# sysctl net.inet.ip.portrange.reservedhigh=0
```

To prevent the `root` user from being affected by this policy, set `security.mac.portacl.suser_exempt` to a non-zero value.

```
# sysctl security.mac.portacl.suser_exempt=1
```

To allow the `www` user with UID 80 to bind to port 80 without ever needing `root` privilege:

```
# sysctl security.mac.portacl.rules=uid:80:tcp:80
```

This next example permits the user with the UID of 1001 to bind to TCP ports 110 (POP3) and 995 (POP3s):

```
# sysctl security.mac.portacl.rules=uid:1001:tcp:110,uid:1001:tcp:995
```

### 18.5.5. The MAC Partition Policy

Module name: mac_partition.ko

Kernel configuration line: `options MAC_PARTITION`

Boot option: `mac_partition_load="YES"`

The mac_partition(4) policy drops processes into specific "partitions" based on their MAC label. Most configuration for this policy is done using setpmac(8). One `sysctl` tunable is available for this policy:

- `security.mac.partition.enabled` enables the enforcement of MAC process partitions.

When this policy is enabled, users will only be permitted to see their processes, and any others within their partition, but will not be permitted to work with utilities outside the scope of this partition. For instance, a user in the `insecure` class will not be permitted to access `top` as well as many other commands that must spawn a process.

This example adds `top` to the label set on users in the `insecure` class. All processes spawned by users in the `insecure` class will stay in the `partition/13` label.

```
# setpmac partition/13 top
```

This command displays the partition label and the process list:

```
# ps Zax
```

This command displays another user's process partition label and that user's currently running processes:

```
# ps -ZU trhodes
```



Users can see processes in `root`'s label unless the mac_seeotheruids(4) policy is loaded.

### 18.5.6. The MAC Multi-Level Security Module

Module name: mac_mls.ko

Kernel configuration line: `options MAC_MLS`

Boot option: `mac_mls_load="YES"`

The mac_mls(4) policy controls access between subjects and objects in the system by enforcing a strict information flow policy.

In MLS environments, a "clearance" level is set in the label of each subject or object, along with compartments. Since these clearance levels can reach numbers greater than several thousand, it would be a daunting task to thoroughly configure every subject or object. To ease this administrative overhead, three labels are included in this policy: `mls/low`, `mls/equal`, and `mls/high`, where:

- Anything labeled with `mls/low` will have a low clearance level and not be permitted to access information of a higher level. This label also prevents objects of a higher clearance level from writing or passing information to a lower level.
- `mls/equal` should be placed on objects which should be exempt from the policy.
- `mls/high` is the highest level of clearance possible. Objects assigned this label will hold dominance over all other objects in the system; however, they will not permit the leaking of information to objects of a lower class.

MLS provides:

- A hierarchical security level with a set of non-hierarchical categories.
- Fixed rules of `no read up, no write down`. This means that a subject can have read access to objects on its own level or below, but not above. Similarly, a subject can have write access to objects on its own level or above, but not beneath.
- Secrecy, or the prevention of inappropriate disclosure of data.
- A basis for the design of systems that concurrently handle data at multiple sensitivity levels

without leaking information between secret and confidential.

The following `sysctl` tunables are available:

- `security.mac.mls.enabled` is used to enable or disable the MLS policy.
- `security.mac.mls.ptys_equal` labels all pty(4) devices as `mls/equal` during creation.
- `security.mac.mls.revocation_enabled` revokes access to objects after their label changes to a label of a lower grade.
- `security.mac.mls.max_compartments` sets the maximum number of compartment levels allowed on a system.

To manipulate MLS labels, use setfmac(8). To assign a label to an object:

```
# setfmac mls/5 test
```

To get the MLS label for the file test:

```
# getfmac test
```

Another approach is to create a master policy file in /etc/ which specifies the MLS policy information and to feed that file to `setfmac`.

When using the MLS policy module, an administrator plans to control the flow of sensitive information. The default `block read up block write down` sets everything to a low state. Everything is accessible and an administrator slowly augments the confidentiality of the information.

Beyond the three basic label options, an administrator may group users and groups as required to block the information flow between them. It might be easier to look at the information in clearance levels using descriptive words, such as classifications of `Confidential`, `Secret`, and `Top Secret`. Some administrators instead create different groups based on project levels. Regardless of the classification method, a well thought out plan must exist before implementing a restrictive policy.

Some example situations for the MLS policy module include an e-commerce web server, a file server holding critical company information, and financial institution environments.

### 18.5.7. The MAC Biba Module

Module name: mac_biba.ko

Kernel configuration line: `options MAC_BIBA`

Boot option: `mac_biba_load="YES"`

The mac_biba(4) module loads the MAC Biba policy. This policy is similar to the MLS policy with the exception that the rules for information flow are slightly reversed. This is to prevent the downward flow of sensitive information whereas the MLS policy prevents the upward flow of sensitive information.

In Biba environments, an "integrity" label is set on each subject or object. These labels are made up of hierarchical grades and non-hierarchical components. As a grade ascends, so does its integrity.

Supported labels are `biba/low`, `biba/equal`, and `biba/high`, where:

- `biba/low` is considered the lowest integrity an object or subject may have. Setting this on objects or subjects blocks their write access to objects or subjects marked as `biba/high`, but will not prevent read access.
- `biba/equal` should only be placed on objects considered to be exempt from the policy.
- `biba/high` permits writing to objects set at a lower label, but does not permit reading that object. It is recommended that this label be placed on objects that affect the integrity of the entire system.

Biba provides:

- Hierarchical integrity levels with a set of non-hierarchical integrity categories.
- Fixed rules are `no write up, no read down`, the opposite of MLS. A subject can have write access to objects on its own level or below, but not above. Similarly, a subject can have read access to objects on its own level or above, but not below.
- Integrity by preventing inappropriate modification of data.
- Integrity levels instead of MLS sensitivity levels.

The following tunables can be used to manipulate the Biba policy:

- `security.mac.biba.enabled` is used to enable or disable enforcement of the Biba policy on the target machine.
- `security.mac.biba.ptys_equal` is used to disable the Biba policy on pty(4) devices.
- `security.mac.biba.revocation_enabled` forces the revocation of access to objects if the label is changed to dominate the subject.

To access the Biba policy setting on system objects, use `setfmac` and `getfmac`:

```
# setfmac biba/low test
# getfmac test
test: biba/low
```

Integrity, which is different from sensitivity, is used to guarantee that information is not manipulated by untrusted parties. This includes information passed between subjects and objects. It ensures that users will only be able to modify or access information they have been given explicit access to. The mac_biba(4) security policy module permits an administrator to configure which files and programs a user may see and invoke while assuring that the programs and files are trusted by the system for that user.

During the initial planning phase, an administrator must be prepared to partition users into grades, levels, and areas. The system will default to a high label once this policy module is enabled, and it is up to the administrator to configure the different grades and levels for users. Instead of using

clearance levels, a good planning method could include topics. For instance, only allow developers modification access to the source code repository, source code compiler, and other development utilities. Other users would be grouped into other categories such as testers, designers, or end users and would only be permitted read access.

A lower integrity subject is unable to write to a higher integrity subject and a higher integrity subject cannot list or read a lower integrity object. Setting a label at the lowest possible grade could make it inaccessible to subjects. Some prospective environments for this security policy module would include a constrained web server, a development and test machine, and a source code repository. A less useful implementation would be a personal workstation, a machine used as a router, or a network firewall.

### 18.5.8. The MAC Low-watermark Module

Module name: mac_lomac.ko

Kernel configuration line: `options MAC_LOMAC`

Boot option: `mac_lomac_load="YES"`

Unlike the MAC Biba policy, the mac_lomac(4) policy permits access to lower integrity objects only after decreasing the integrity level to not disrupt any integrity rules.

The Low-watermark integrity policy works almost identically to Biba, with the exception of using floating labels to support subject demotion via an auxiliary grade compartment. This secondary compartment takes the form `[auxgrade]`. When assigning a policy with an auxiliary grade, use the syntax `lomac/10[2]`, where `2` is the auxiliary grade.

This policy relies on the ubiquitous labeling of all system objects with integrity labels, permitting subjects to read from low integrity objects and then downgrading the label on the subject to prevent future writes to high integrity objects using `[auxgrade]`. The policy may provide greater compatibility and require less initial configuration than Biba.

Like the Biba and MLS policies, `setfmac` and `setpmac` are used to place labels on system objects:

```
# setfmac /usr/home/trhodes lomac/high[low]
# getfmac /usr/home/trhodes lomac/high[low]
```

The auxiliary grade `low` is a feature provided only by the MACLOMAC policy.

## 18.6. User Lock Down

This example considers a relatively small storage system with fewer than fifty users. Users will have login capabilities and are permitted to store data and access resources.

For this scenario, the mac_bsdextended(4) and mac_seeotheruids(4) policy modules could co-exist and block access to system objects while hiding user processes.

Begin by adding the following line to /boot/loader.conf:

```
mac_seeotheruids_load="YES"
```

The mac_bsdextended(4) security policy module may be activated by adding this line to /etc/rc.conf:

```
ugidfw_enable="YES"
```

Default rules stored in /etc/rc.bsdextended will be loaded at system initialization. However, the default entries may need modification. Since this machine is expected only to service users, everything may be left commented out except the last two lines in order to force the loading of user owned system objects by default.

Add the required users to this machine and reboot. For testing purposes, try logging in as a different user across two consoles. Run `ps aux` to see if processes of other users are visible. Verify that running ls(1) on another user's home directory fails.

Do not try to test with the `root` user unless the specific `sysctl`s have been modified to block super user access.



When a new user is added, their mac_bsdextended(4) rule will not be in the ruleset list. To update the ruleset quickly, unload the security policy module and reload it again using kldunload(8) and kldload(8).

# 18.7. Nagios in a MAC Jail

This section demonstrates the steps that are needed to implement the Nagios network monitoring system in a MAC environment. This is meant as an example which still requires the administrator to test that the implemented policy meets the security requirements of the network before using in a production environment.

This example requires `multilabel` to be set on each file system. It also assumes that net-mgmt/nagios-plugins, net-mgmt/nagios, and www/apache22 are all installed, configured, and working correctly before attempting the integration into the MAC framework.

## 18.7.1. Create an Insecure User Class

Begin the procedure by adding the following user class to /etc/login.conf:

```
insecure:\
:welcome=/etc/motd:\
:setenv=MAIL=/var/mail/$,BLOCKSIZE=K:\
:path=~/bin:/sbin:/bin:/usr/sbin:/usr/bin:/usr/local/sbin:/usr/local/bin
:manpath=/usr/share/man /usr/local/man:\
:nologin=/usr/sbin/nologin:\
:cputime=1h30m:\
:datasize=8M:\
:vmemoryuse=100M:\
```

```
:stacksize=2M:\
:memorylocked=4M:\
:memoryuse=8M:\
:filesize=8M:\
:coredumpsize=8M:\
:openfiles=24:\
:maxproc=32:\
:priority=0:\
:requirehome:\
:passwordtime=91d:\
:umask=022:\
:ignoretime@:\
:label=biba/10(10-10):
```

Then, add the following line to the default user class section:

```
:label=biba/high:
```

Save the edits and issue the following command to rebuild the database:

```
# cap_mkdb /etc/login.conf
```

### 18.7.2. Configure Users

Set the `root` user to the default class using:

```
# pw usermod root -L default
```

All user accounts that are not `root` will now require a login class. The login class is required, otherwise users will be refused access to common commands. The following `sh` script should do the trick:

```
# for x in `awk -F: '($3 >= 1001) && ($3 != 65534) { print $1 }' \
    /etc/passwd`; do pw usermod $x -L default; done;
```

Next, drop the `nagios` and `www` accounts into the insecure class:

```
# pw usermod nagios -L insecure
# pw usermod www -L insecure
```

### 18.7.3. Create the Contexts File

A contexts file should now be created as /etc/policy.contexts:

```
# This is the default BIBA policy for this system.

# System:
/var/run(/.*)?          biba/equal

/dev/(/.*)?         biba/equal

/var                biba/equal
/var/spool(/.*)?        biba/equal

/var/log(/.*)?          biba/equal

/tmp(/.*)?          biba/equal
/var/tmp(/.*)?          biba/equal

/var/spool/mqueue       biba/equal
/var/spool/clientmqueue     biba/equal

# For Nagios:
/usr/local/etc/nagios(/.*)? biba/10

/var/spool/nagios(/.*)?     biba/10

# For apache
/usr/local/etc/apache(/.*)? biba/10
```

This policy enforces security by setting restrictions on the flow of information. In this specific configuration, users, including `root`, should never be allowed to access Nagios. Configuration files and processes that are a part of Nagios will be completely self contained or jailed.

This file will be read after running `setfsmac` on every file system. This example sets the policy on the root file system:

```
# setfsmac -ef /etc/policy.contexts /
```

Next, add these edits to the main section of /etc/mac.conf:

```
default_labels file ?biba
default_labels ifnet ?biba
default_labels process ?biba
default_labels socket ?biba
```

### 18.7.4. Loader Configuration

To finish the configuration, add the following lines to /boot/loader.conf:

```
mac_biba_load="YES"
mac_seeotheruids_load="YES"
security.mac.biba.trust_all_interfaces=1
```

And the following line to the network card configuration stored in /etc/rc.conf. If the primary network configuration is done via DHCP, this may need to be configured manually after every system boot:

```
maclabel biba/equal
```

### 18.7.5. Testing the Configuration

First, ensure that the web server and Nagios will not be started on system initialization and reboot. Ensure that `root` cannot access any of the files in the Nagios configuration directory. If `root` can list the contents of /var/spool/nagios, something is wrong. Instead, a "permission denied" error should be returned.

If all seems well, Nagios, Apache, and Sendmail can now be started:

```
# cd /etc/mail && make stop && \
setpmac biba/equal make start && setpmac biba/10\(10-10\) apachectl start && \
setpmac biba/10\(10-10\) /usr/local/etc/rc.d/nagios.sh forcestart
```

Double check to ensure that everything is working properly. If not, check the log files for error messages. If needed, use sysctl(8) to disable the mac_biba(4) security policy module and try starting everything again as usual.



The `root` user can still change the security enforcement and edit its configuration files. The following command will permit the degradation of the security policy to a lower grade for a newly spawned shell:

```
# setpmac biba/10 csh
```

To block this from happening, force the user into a range using login.conf(5). If setpmac(8) attempts to run a command outside of the compartment's range, an error will be returned and the command will not be executed. In this case, set root to `biba/high(high-high)`.

## 18.8. Troubleshooting the MAC Framework

This section discusses common configuration errors and how to resolve them.

**The `multilabel` flag does not stay enabled on the root (/) partition**

The following steps may resolve this transient error:

1. Edit /etc/fstab and set the root partition to `ro` for read-only.
2. Reboot into single user mode.
3. Run `tunefs -l enable` on /.
4. Reboot the system.
5. Run `mount -urw`/ and change the `ro` back to `rw` in /etc/fstab and reboot the system again.
6. Double-check the output from `mount` to ensure that `multilabel` has been properly set on the root file system.

#### After establishing a secure environment with MAC, Xorg no longer starts

This could be caused by the MAC `partition` policy or by a mislabeling in one of the MAC labeling policies. To debug, try the following:

1. Check the error message. If the user is in the `insecure` class, the `partition` policy may be the culprit. Try setting the user's class back to the `default` class and rebuild the database with `cap_mkdb`. If this does not alleviate the problem, go to step two.
2. Double-check that the label policies are set correctly for the user, Xorg, and the /dev entries.
3. If neither of these resolve the problem, send the error message and a description of the environment to the FreeBSD general questions mailing list.

#### The `_secure_path: unable to stat .login_conf` error appears

This error can appear when a user attempts to switch from the `root` user to another user in the system. This message usually occurs when the user has a higher label setting than that of the user they are attempting to become. For instance, if `joe` has a default label of `biba/low` and `root` has a label of `biba/high`, `root` cannot view `joe`'s home directory. This will happen whether or not `root` has used `su` to become `joe` as the Biba integrity model will not permit `root` to view objects set at a lower integrity level.

#### The system no longer recognizes `root`

When this occurs, `whoami` returns `0` and `su` returns `who are you?`.

This can happen if a labeling policy has been disabled by sysctl(8) or the policy module was unloaded. If the policy is disabled, the login capabilities database needs to be reconfigured. Double check /etc/login.conf to ensure that all `label` options have been removed and rebuild the database with `cap_mkdb`.

This may also happen if a policy restricts access to master.passwd. This is usually caused by an administrator altering the file under a label which conflicts with the general policy being used by the system. In these cases, the user information would be read by the system and access would be blocked as the file has inherited the new label. Disable the policy using sysctl(8) and everything should return to normal.

# Chapter 19. Security Event Auditing

## 19.1. Synopsis

The FreeBSD operating system includes support for security event auditing. Event auditing supports reliable, fine-grained, and configurable logging of a variety of security-relevant system events, including logins, configuration changes, and file and network access. These log records can be invaluable for live system monitoring, intrusion detection, and postmortem analysis. FreeBSD implements Sun™'s published Basic Security Module (BSM) Application Programming Interface (API) and file format, and is interoperable with the Solaris™ and Mac OS® X audit implementations.

This chapter focuses on the installation and configuration of event auditing. It explains audit policies and provides an example audit configuration.

Read this chapter to learn:

- What event auditing is and how it works.
- How to configure event auditing on FreeBSD for users and processes.
- How to review the audit trail using the audit reduction and review tools.

Before reading this chapter:

- Understand UNIX® and FreeBSD basics (FreeBSD Basics).
- Be familiar with the basics of kernel configuration/compilation (Configuring the FreeBSD Kernel).
- Have some familiarity with security and how it pertains to FreeBSD (Security).



The audit facility has some known limitations. Not all security-relevant system events are auditable and some login mechanisms, such as Xorg-based display managers and third-party daemons, do not properly configure auditing for user login sessions.

The security event auditing facility is able to generate very detailed logs of system activity. On a busy system, trail file data can be very large when configured for high detail, exceeding gigabytes a week in some configurations. Administrators should take into account the disk space requirements associated with high volume audit configurations. For example, it may be desirable to dedicate a file system to /var/audit so that other file systems are not affected if the audit file system becomes full.

## 19.2. Key Terms

The following terms are related to security event auditing:

- *event*: an auditable event is any event that can be logged using the audit subsystem. Examples of security-relevant events include the creation of a file, the building of a network connection, or a

user logging in. Events are either "attributable", meaning that they can be traced to an authenticated user, or "non-attributable". Examples of non-attributable events are any events that occur before authentication in the login process, such as bad password attempts.

- *class*: a named set of related events which are used in selection expressions. Commonly used classes of events include "file creation" (fc), "exec" (ex), and "login_logout" (lo).
- *record*: an audit log entry describing a security event. Records contain a record event type, information on the subject (user) performing the action, date and time information, information on any objects or arguments, and a success or failure condition.
- *trail*: a log file consisting of a series of audit records describing security events. Trails are in roughly chronological order with respect to the time events completed. Only authorized processes are allowed to commit records to the audit trail.
- *selection expression*: a string containing a list of prefixes and audit event class names used to match events.
- *preselection*: the process by which the system identifies which events are of interest to the administrator. The preselection configuration uses a series of selection expressions to identify which classes of events to audit for which users, as well as global settings that apply to both authenticated and unauthenticated processes.
- *reduction*: the process by which records from existing audit trails are selected for preservation, printing, or analysis. Likewise, the process by which undesired audit records are removed from the audit trail. Using reduction, administrators can implement policies for the preservation of audit data. For example, detailed audit trails might be kept for one month, but after that, trails might be reduced in order to preserve only login information for archival purposes.

## 19.3. Audit Configuration

User space support for event auditing is installed as part of the base FreeBSD operating system. Kernel support is available in the GENERIC kernel by default, and auditd(8) can be enabled by adding the following line to /etc/rc.conf:

```
auditd_enable="YES"
```

Then, start the audit daemon:

```
# service auditd start
```

Users who prefer to compile a custom kernel must include the following line in their custom kernel configuration file:

```
options AUDIT
```

### 19.3.1. Event Selection Expressions

Selection expressions are used in a number of places in the audit configuration to determine which events should be audited. Expressions contain a list of event classes to match. Selection expressions are evaluated from left to right, and two expressions are combined by appending one onto the other.

Default Audit Event Classes summarizes the default audit event classes:

*Table 32. Default Audit Event Classes*

| Class Name | Description | Action |
|---|---|---|
| all | all | Match all event classes. |
| aa | authentication and authorization | |
| ad | administrative | Administrative actions performed on the system as a whole. |
| ap | application | Application defined action. |
| cl | file close | Audit calls to the `close` system call. |
| ex | exec | Audit program execution. Auditing of command line arguments and environmental variables is controlled via audit_control(5) using the `argv` and `envv` parameters to the `policy` setting. |
| fa | file attribute access | Audit the access of object attributes such as stat(1) and pathconf(2). |
| fc | file create | Audit events where a file is created as a result. |
| fd | file delete | Audit events where file deletion occurs. |
| fm | file attribute modify | Audit events where file attribute modification occurs, such as by chown(8), chflags(1), and flock(2). |
| fr | file read | Audit events in which data is read or files are opened for reading. |

| Class Name | Description | Action |
|---|---|---|
| fw | file write | Audit events in which data is written or files are written or modified. |
| io | ioctl | Audit use of the `ioctl` system call. |
| ip | ipc | Audit various forms of Inter-Process Communication, including POSIX pipes and System V IPC operations. |
| lo | login_logout | Audit login(1) and logout(1) events. |
| na | non attributable | Audit non-attributable events. |
| no | invalid class | Match no audit events. |
| nt | network | Audit events related to network actions such as connect(2) and accept(2). |
| ot | other | Audit miscellaneous events. |
| pc | process | Audit process operations such as exec(3) and exit(3). |

These audit event classes may be customized by modifying the audit_class and audit_event configuration files.

Each audit event class may be combined with a prefix indicating whether successful/failed operations are matched, and whether the entry is adding or removing matching for the class and type. Prefixes for Audit Event Classes summarizes the available prefixes:

*Table 33. Prefixes for Audit Event Classes*

| Prefix | Action |
|---|---|
| + | Audit successful events in this class. |
| - | Audit failed events in this class. |
| ^ | Audit neither successful nor failed events in this class. |
| ^+ | Do not audit successful events in this class. |
| ^- | Do not audit failed events in this class. |

If no prefix is present, both successful and failed instances of the event will be audited.

The following example selection string selects both successful and failed login/logout events, but only successful execution events:

```
lo,+ex
```

### 19.3.2. Configuration Files

The following configuration files for security event auditing are found in /etc/security:

- audit_class: contains the definitions of the audit classes.
- audit_control: controls aspects of the audit subsystem, such as default audit classes, minimum disk space to leave on the audit log volume, and maximum audit trail size.
- audit_event: textual names and descriptions of system audit events and a list of which classes each event is in.
- audit_user: user-specific audit requirements to be combined with the global defaults at login.
- audit_warn: a customizable shell script used by auditd(8) to generate warning messages in exceptional situations, such as when space for audit records is running low or when the audit trail file has been rotated.



Audit configuration files should be edited and maintained carefully, as errors in configuration may result in improper logging of events.

In most cases, administrators will only need to modify audit_control and audit_user. The first file controls system-wide audit properties and policies and the second file may be used to fine-tune auditing by user.

#### 19.3.2.1. The audit_control File

A number of defaults for the audit subsystem are specified in audit_control:

```
dir:/var/audit
dist:off
flags:lo,aa
minfree:5
naflags:lo,aa
policy:cnt,argv
filesz:2M
expire-after:10M
```

The `dir` entry is used to set one or more directories where audit logs will be stored. If more than one directory entry appears, they will be used in order as they fill. It is common to configure audit so that audit logs are stored on a dedicated file system, in order to prevent interference between the audit subsystem and other subsystems if the file system fills.

If the `dist` field is set to `on` or `yes`, hard links will be created to all trail files in /var/audit/dist.

The `flags` field sets the system-wide default preselection mask for attributable events. In the example above, successful and failed login/logout events as well as authentication and

authorization are audited for all users.

The `minfree` entry defines the minimum percentage of free space for the file system where the audit trail is stored.

The `naflags` entry specifies audit classes to be audited for non-attributed events, such as the login/logout process and authentication and authorization.

The `policy` entry specifies a comma-separated list of policy flags controlling various aspects of audit behavior. The `cnt` indicates that the system should continue running despite an auditing failure (this flag is highly recommended). The other flag, `argv`, causes command line arguments to the execve(2) system call to be audited as part of command execution.

The `filesz` entry specifies the maximum size for an audit trail before automatically terminating and rotating the trail file. A value of `0` disables automatic log rotation. If the requested file size is below the minimum of 512k, it will be ignored and a log message will be generated.

The `expire-after` field specifies when audit log files will expire and be removed.

#### 19.3.2.2. The audit_user File

The administrator can specify further audit requirements for specific users in audit_user. Each line configures auditing for a user via two fields: the `alwaysaudit` field specifies a set of events that should always be audited for the user, and the `neveraudit` field specifies a set of events that should never be audited for the user.

The following example entries audit login/logout events and successful command execution for `root` and file creation and successful command execution for `www`. If used with the default audit_control, the `lo` entry for `root` is redundant, and login/logout events will also be audited for `www`.

```
root:lo,+ex:no
www:fc,+ex:no
```

## 19.4. Working with Audit Trails

Since audit trails are stored in the BSM binary format, several built-in tools are available to modify or convert these trails to text. To convert trail files to a simple text format, use `praudit`. To reduce the audit trail file for analysis, archiving, or printing purposes, use `auditreduce`. This utility supports a variety of selection parameters, including event type, event class, user, date or time of the event, and the file path or object acted on.

For example, to dump the entire contents of a specified audit log in plain text:

```
# praudit /var/audit/AUDITFILE
```

Where *AUDITFILE* is the audit log to dump.